

# VulATMHD: Joint adaptive triplet mining and hybrid distillation for type-aware vulnerability classification

Xuanye Wang<sup>a</sup>, Lu Lu<sup>a,b</sup> \*

<sup>a</sup> School of Computer Science and Engineering, South China University of Technology, Guangzhou, 510006, Guangdong, China

<sup>b</sup> Peng Cheng Laboratory, Shenzhen, 518055, Guangdong, China

## ARTICLE INFO

Dataset link: <https://github.com/wxyzjp123/VulATMHD>

### Keywords:

Software vulnerability  
Adaptive triplet mining  
Hybrid distillation

## ABSTRACT

**Context:** Vulnerability detection leveraging pre-trained models has achieved notable success, but its coarse-grained outputs fail to provide security engineers with vulnerability type information. Recent type-aware Software Vulnerability Classification (SVC) methods mitigate this gap, but often neglect inter-type semantic relationships and exhibit limited knowledge transfer, resulting in suboptimal learned representations.

**Objective:** To address these limitations, this study proposes VulATMHD, a novel type-aware SVC framework that integrates adaptive triplet mining with hybrid distillation.

**Methods:** VulATMHD first groups vulnerability types based on common weakness enumeration abstract types. It then constructs a multi-teacher architecture, with each teacher assigned to a specific group. Adaptive triplet mining is introduced to guide feature learning, yielding feature representations that are intra-class compact and inter-class separable. Since each teacher is optimized for intra-group performance, VulATMHD further introduces a hybrid distillation strategy to transfer both feature representations and label distributions from the teacher ensemble to a pre-trained student.

**Results:** Empirical evaluations on the BigVul dataset show that, compared to baseline methods, VulATMHD improves Accuracy and weighted F1-score by 4.7%–29.9% and 5.7%–34.1%, respectively. Moreover, VulATMHD is compatible with various pre-trained models, such as CodeBERT, CodeT5+, and GraphCodeBERT.

**Conclusion:** The proposed VulATMHD outperforms state-of-the-art SVC methods and exhibits superior robustness and scalability in downstream tasks, highlighting its potential for practical applications.

## 1. Introduction

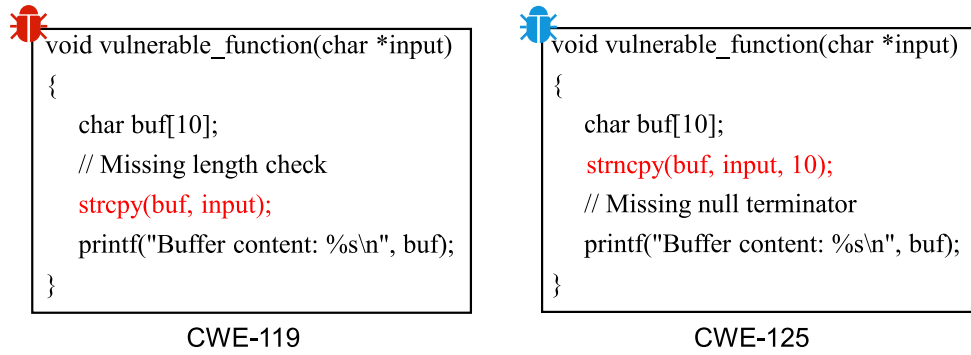
Software vulnerabilities are security weaknesses that arise during software design, implementation, or configuration [1]. Recently, the U.S. National Public Data Corporation suffered a cyberattack exploiting vulnerabilities, resulting in the breach of 2.9 billion sensitive records and ultimately forcing the company into bankruptcy [2]. To standardize the classification and description of these vulnerabilities, the community widely adopts the Common Weakness Enumeration (CWE) framework [3]. By 2015, CWE had cataloged 6494 vulnerabilities, and with the rapid expansion of modern software, this number soared to 40,301 by 2024 [4]. Consequently, identifying potential vulnerabilities is a critical task for development teams to ensure software security.

With the rapid development of pre-trained models, various pre-trained model-based Software Vulnerability Detection (SVD) methods have been proposed to determine whether a given sample contains vulnerabilities. Trained on large-scale code corpora, these models capture rich semantic representations, facilitating effective transfer to the SVD

task. Fu et al. [5] proposed LineVul, and Liu et al. [6] proposed PM2-CNN, which utilize CodeBERT [7] and UniXcoder [8], respectively, to extract deep semantic features from the source code, significantly outperforming previous graph-based SVD methods [9,10].

Despite significant progress in SVD research, the coarse-grained outputs of these methods still offer limited utility to security engineers. In practice, engineers often face a backlog of vulnerability alerts generated by automated detection tools. Without fine-grained vulnerability information (e.g., vulnerability types), they must manually inspect and prioritize each alert, which increases the risk of critical vulnerabilities being exploited. For example, CWE-125 (Out-of-bounds Read) is considered a critical vulnerability. Improper bounds checking may allow attackers to access sensitive memory regions, leading to information disclosure. In contrast, CWE-310 (Cryptographic Issues) is less severe. It may weaken data protection but does not allow direct system control. Therefore, recent studies have explored various directions to bridge this gap, such as explainability methods that highlight vulnerability

\* Corresponding author at: School of Computer Science and Engineering, South China University of Technology, Guangzhou, 510006, Guangdong, China.  
E-mail addresses: [202310188991@mail.scut.edu.cn](mailto:202310188991@mail.scut.edu.cn) (X. Wang), [lul@scut.edu.cn](mailto:lul@scut.edu.cn) (L. Lu).



**Fig. 1.** An example illustrating the subtle differences between CWE-119 and CWE-125. The left example, using `strcpy` without length checking, may cause a buffer overflow. In the right example, `strcpy` limits the write length, but missing a null terminator may still lead to out-of-bounds reads.

locations or potential causes [11,12] and Software Vulnerability Classification (SVC) methods [13–15] that identify vulnerability severities or types. This study focuses on type-aware SVC, which provides specific vulnerability type information critical to real-world remediation efforts.

Recently proposed type-aware SVC methods have focused on addressing the long-tailed distribution of vulnerability types (i.e., a phenomenon where most samples belong to a few common types, while many types are rare). Fu et al. [14] proposed VulExplainer, which introduces hierarchical learning to address long-tailed label distribution and applies label distillation to transfer knowledge into a pre-trained model. Wen et al. [15] introduced LIVABLE, which incorporates an adaptive re-weighting module to balance attention between common and rare classes. While these methods demonstrate certain effectiveness, two key limitations remain: (1) *Semantic overlap among vulnerability types* — Several vulnerability types share significant semantic similarities in their code patterns. Fig. 1 illustrates an example involving CWE-119 (Buffer Errors) and CWE-125, which differ solely in memory access direction. However, existing SVC methods focus on balancing attention across different types but neglect their fine-grained semantic relationships, leading to overlapping class boundaries. Although several SVD methods [16,17] employ contrastive learning to enlarge the feature distance between vulnerable and non-vulnerable features, SVC is more challenging due to its larger number of types. This increase also raises the prevalence of outliers, resulting in noisy feature distances during early training [18,19] and impairing effective sample mining. (2) *Insufficient knowledge transfer* — They exhibit limited knowledge transfer, yielding suboptimal learned representations. For example, VulExplainer performs knowledge distillation only at the label level, neglecting the semantic features captured by teachers. In contrast, LIVABLE replaces pre-trained models with a graph neural network, thus forfeiting the rich representations acquired through large-scale pre-training.

To address these limitations, this study proposes VulATMHD, a novel type-aware SVC framework that integrates adaptive triplet mining and hybrid distillation. The framework operates in two stages: first, it divides all vulnerability types into five groups based on CWE abstract types [20] for balanced distribution. Then, it employs a multi-teacher architecture, with each teacher is trained to classify the types within its domain. To capture the fine-grained semantic relationships between vulnerability types, adaptive triplet mining is introduced to optimize the teacher ensemble. This strategy extends the batch-hard triplet mining [21] by incorporating CWE abstract type information as semantic prior knowledge. Its objective is to learn a feature space where semantically similar types within the same domain are well-separated, while samples of the same type form compact clusters. Second, as each teacher is tailored to its respective group, a pre-trained model serves as the student to unify the group-specific knowledge. This student is further optimized via a hybrid distillation strategy that aligns both feature representations and label distributions from the teacher ensemble, facilitating more effective knowledge transfer.

An empirical evaluation of VulATMHD is conducted on the BigVul [22] dataset, which contains 8636 vulnerability samples covering 44 vulnerability types [22]. Experimental results indicate that VulATMHD outperforms baseline methods, including VulExplainer and LIVABLE, with improvements of 4.7%–29.9% in Accuracy and 5.7%–34.1% in weighted F1-score. Beyond classification performance, VulATMHD is further evaluated against two adversarial attacks [23,24], demonstrating superior robustness compared to baseline methods. In addition, to evaluate its scalability, VulATMHD is transferred to the SVD task and evaluated on the BigVul [22], Devign [9], and Reveal [10] datasets. It achieves competitive performance, outperforming representative SVD methods such as LineVul [5] and PM2-CNN [6].

In summary, the main contributions of our VulATMHD are as follows:

1. VulATMHD introduces adaptive triplet mining to capture fine-grained semantic relationships among vulnerability types, thereby enabling the teacher ensemble to learn compact intra-type and well-separated inter-type features.
2. VulATMHD designs a hybrid distillation strategy that jointly aligns label semantics and feature representations between the teacher ensemble and the student, enabling more comprehensive knowledge transfer.
3. Empirical evaluation on the BigVul dataset demonstrates that VulATMHD significantly outperforms baseline methods in both Accuracy and weighted F1-score. In addition, it exhibits superior robustness and scalability across downstream tasks.

The dataset and models used in this study are publicly available at <https://github.com/wxyzjp123/VulATMHD>.

The remainder of this paper is organized as follows. Section 2 introduces representative studies on SVD and SVC. Section 3 presents the preliminaries. Section 4 describes the details of the proposed VulATMHD framework. Section 5 describes the experimental setup. Section 6 analyzes the experimental results. Section 7 offers supplementary discussion. Section 8 discusses the threats to validity. Section 9 concludes this study.

## 2. Related work

This section provides a comprehensive review of representative studies on SVD and type-aware SVC.

### 2.1. Software vulnerability detection

The goal of SVD is to determine whether a sample contains vulnerabilities. To capture richer semantics, several SVD methods construct program graphs and employ graph neural networks to encode syntactic and semantic dependencies. Zhou et al. [9] proposed Devign and Chakraborty et al. [10] proposed Reveal, both of which represent

source code as graphs and employ Gated Graph Recurrent Networks (GGRN) and Gated Graph Neural Networks (GGNN), respectively, to capture vulnerability patterns. Nguyen et al. [25] proposed ReGVD, which transforms a sequence of code tokens into a graph and uses a Graph Convolutional Network (GCN) with residual connections to learn code representations. Recent advances in SVD leverage pre-trained models, which effectively capture vulnerability-related semantics via representation learning on large-scale code corpora. Fu et al. [5] introduced LineVul, which applies CodeBERT to capture vulnerability patterns. Similarly, Liu et al. [6] presented PM2-CNN, which integrates semantic and textual features using UniXcoder, and employs TextCNN [26] for detection. Yu et al. [27] proposed a semi-supervised method that selects pseudo-labels via parameter information gain and achieves performance comparable to LineVul with limited labeled data.

While pre-trained model-based SVD methods have achieved remarkable success, they are limited in providing detailed type information, leaving security engineers without the insights needed for effective prioritization and remediation of vulnerabilities.

## 2.2. Type-aware software vulnerability classification

Type-aware SVC aims to assign a specific vulnerability type (i.e., CWE-ID) to each vulnerability sample. Early research classified vulnerabilities based on their textual reports. Wang et al. [28] and Vishnu et al. [29] adopted BERT-based and self-attention-based models to classify vulnerabilities from textual descriptions into common CWE-IDs. Nevertheless, such descriptions are often unavailable in early development when SVC is most needed. In response, recent research has shifted toward classifying vulnerability samples based on code semantics. Leveraging SVD models' semantic feature extraction, Lu et al. [30] adapted them for SVC by replacing output layers with multi-class classification heads for five CWE-IDs. Xu et al. [31] utilized a graph neural network to extract semantic and structural features, supporting the classification of eight smart contract vulnerability types. However, these methods cover only a limited number of types. Class imbalance from long-tailed label distribution becomes more severe as the number of target types increases. Fu et al. [14] proposed VulExplainer, which performs hierarchical learning across 44 CWE-IDs and employs label distillation to transfer classification knowledge to a pre-trained model. Wen et al. [15] proposed LIVABLE, which introduces an adaptive re-weighting module that adjusts the learning weights for different CWE-IDs based on their sample sizes and training epochs, thereby alleviating the impact of long-tailed distributions.

While VulExplainer and LIVABLE have mitigated the impact of long-tailed label distributions, they neglect fine-grained semantic relationships among vulnerability types and demonstrate limited knowledge transfer. These limitations motivate the proposal of VulATMHD.

## 3. Preliminaries

This section presents the preliminaries necessary for understanding VulATMHD.

### 3.1. Batch-hard triplet mining

Batch-hard triplet mining is a sample mining strategy [21] used to construct triplets that enhances feature discriminability by compressing intra-class distributions and increasing inter-class separability.

Specifically, each sample in a mini-batch is treated in turn as an anchor, denoted as  $\mathbf{h}_i$ , which is paired with a hardest positive  $\mathbf{h}_{p^*}$  (i.e., the farthest sample from the same class) and a hardest negative  $\mathbf{h}_{n^*}$  (i.e., the closest sample from a different class), as defined below:

$$\mathbf{h}_{p^*} = \arg \max_{p \in \mathcal{P}(i)} D_{ip}, \quad \mathbf{h}_{n^*} = \arg \min_{n \in \mathcal{N}(i)} D_{in}, \quad (1)$$

where  $\mathcal{P}(i)$  and  $\mathcal{N}(i)$  denote the sets of positive and negative samples with respect to the anchor  $\mathbf{h}_i$ , respectively, and  $D_{ip}$  ( $D_{in}$ ) represents the

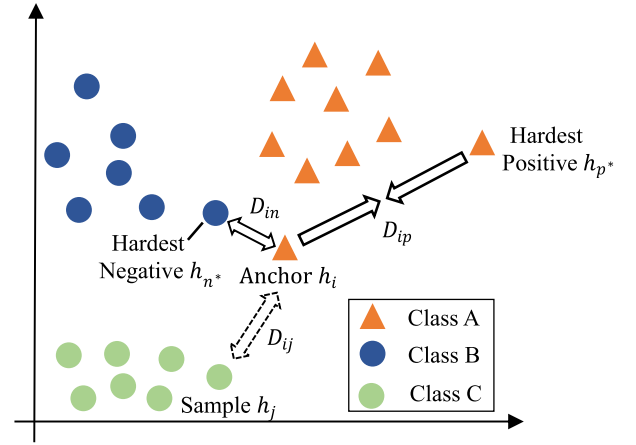


Fig. 2. An example of hardest positive and negative sample selection under batch-hard triplet mining. Class A contains the anchor  $\mathbf{h}_i$  and its positives  $\mathcal{P}(i)$ ; Classes B and C provide negatives  $\mathcal{N}(i) = \mathcal{B} \cup \mathcal{C}$ . The sample  $\mathbf{h}_{p^*}$  is selected as the hardest positive due to being the farthest in-class sample from anchor  $\mathbf{h}_i$ , while  $\mathbf{h}_{n^*}$  from Class B is selected as the hardest negative since  $D_{ij} > D_{in^*}$ .

feature distance between  $\mathbf{h}_i$  and a positive sample  $\mathbf{h}_p$  (a negative sample  $\mathbf{h}_n$ ). Fig. 2 illustrates the selection of  $\mathbf{h}_{p^*}$  and  $\mathbf{h}_{n^*}$  for a given  $\mathbf{h}_i$  in a toy three-class example. Then, the triplet loss for each anchor is defined as:

$$\mathcal{L}_{\text{trip}}(\mathbf{h}_i) = \max \left\{ D_{ip}^* - D_{in^*} + m, 0 \right\}, \quad (2)$$

where  $m$  is a predefined margin that enforces a minimum distance gap between positive and negative pairs. This loss encourages the hardest positive to be closer than the hardest negative by at least a margin  $m$ . When this condition is violated, the model is updated to pull the hardest positive closer to  $\mathbf{h}_i$  and push the hardest negative farther away, thus promoting a more discriminative embedding space.

### 3.2. Knowledge distillation

Knowledge distillation is a transfer learning strategy in which a more powerful teacher model is used to supervise the training of a student model, with the goal of transferring the knowledge embedded in the teacher to the student.

A common strategy of knowledge distillation is label distillation [32], in which the student learns to mimic the teacher's behavior by minimizing the divergence between their output distributions. Specifically, the teacher produces a label distribution for each sample using the softmax function. The Kullback–Leibler (KL) divergence [33] is then used to quantify the distance between the teacher and student outputs, as defined by:

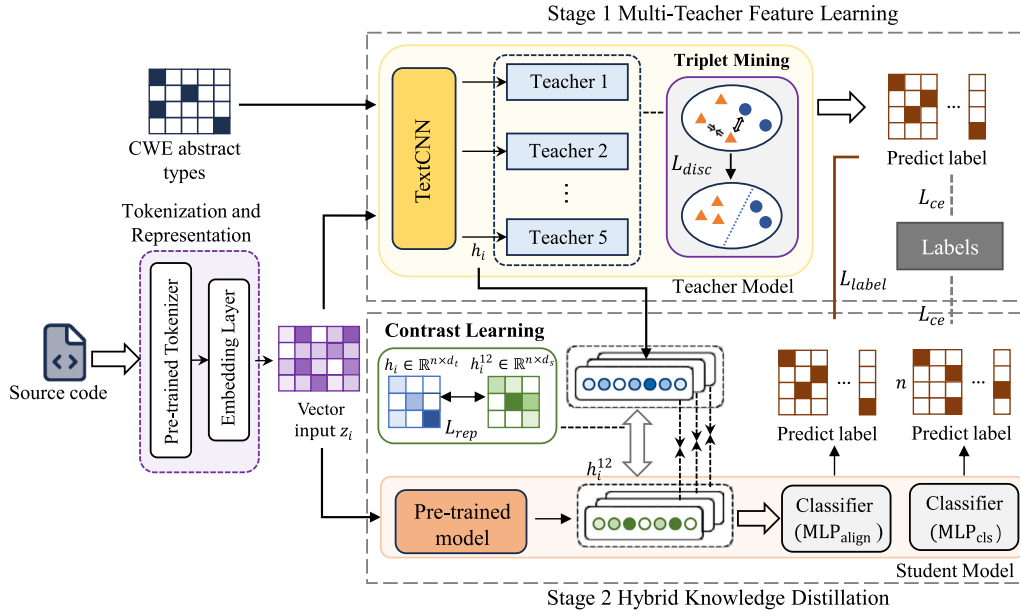
$$\mathcal{L}_{\text{label}} = \text{KL}(\mathbf{y}^t \parallel \mathbf{y}^s) = \sum_{c=1}^{|Y|} \mathbf{y}_c^t \log \left( \frac{\mathbf{y}_c^t}{\mathbf{y}_c^s} \right), \quad (3)$$

where  $\mathbf{y}^t$  and  $\mathbf{y}^s$  denote the predicted distributions of the teacher and the student, respectively,  $\mathbf{y}_c^t$  and  $\mathbf{y}_c^s$  represent the predicted probabilities for class  $c$ , and  $|Y|$  is the total number of classes.

### 3.3. CWE abstract types

Proposed by the CWE Community [20], CWE abstract types are high-level semantic labels, rather than specific vulnerabilities. Each CWE-ID is associated with exactly one CWE abstract type, which organizes related vulnerability types into hierarchical groups based on shared characteristics.

The BigVul dataset used in this study includes five CWE abstract types: Class, Base, Category, Variant, and Deprecated. The Class type



**Fig. 3.** The overall architecture of VulATMHD. It first encodes vulnerability samples into vectors and then performs a two-stage process. Stage 1 constructs a multi-teacher ensemble based on CWE abstract types and optimizes it using  $\mathcal{L}_{disc}$  with adaptive triplet mining, along with  $\mathcal{L}_{ce}$ . Stage 2 introduces hybrid distillation by aligning  $h_i$  and  $h_i^{1,2}$  for  $\mathcal{L}_{rep}$ , and uses  $MLP_{align}$  and  $MLP_{cls}$  to compute  $\mathcal{L}_{label}$  and  $\mathcal{L}_{ce}$ .

describes weaknesses in a highly abstract manner, independent of specific languages or technologies. The Base type is more specific, providing sufficient detail to support detection methods. The Category type serves as an organizational tool, grouping weaknesses based on similar attributes. The Variant type is highly specific, targeting particular technologies or languages. The Deprecated type refers to CWE entries that have been retired or superseded. Further information on CWE abstract types is available in [20].

#### 4. VulATMHD framework

To overcome the two key limitations of existing SVC methods, this study proposes VulATMHD, a novel SVC framework that combines adaptive triplet mining with hybrid distillation to enhance semantic differentiation between vulnerability types and improve knowledge transfer. An overview of VulATMHD is shown in Fig. 3.

##### 4.1. Problem definition

Type-aware SVC is formulated as a multi-class classification task that aims to predict the specific vulnerability type associated with each sample. Given a training set  $\mathcal{T} = \{(x_i, y_i)\}_{i=1}^N$ , where  $x_i \in X$  denotes the  $i$ th vulnerability sample, and  $y_i \in Y$  is its target vulnerability type ( $|Y| = 44$ ). The objective is to train a neural network  $f : X \rightarrow Y$  that minimizes the classification loss, which can be expressed as follows:

$$\min_f \frac{1}{N} \sum_{(x_i, y_i) \in \mathcal{T}} \mathcal{L}(f(x_i), y_i), \quad (4)$$

where  $\mathcal{L}(\cdot)$  denotes the classification loss and  $f(x_i)$  denotes the prediction of the neural network for the  $i$ th sample. In this way, the neural network parameters are optimized to achieve better performance on the SVC task.

##### 4.2. Tokenization and representation

Since neural networks require vectorized inputs, VulATMHD first tokenizes each vulnerability sample using the student's pre-trained tokenizer, and then maps the tokens to high-dimensional embeddings. For clarity, CodeBERT is adopted as the student across this section.

CodeBERT employs a byte-pair encoding tokenizer [34] trained on large-scale code corpora. It breaks unseen or complex identifiers into subword units from its vocabulary, thereby mitigating information loss caused by out-of-vocabulary terms. Given a sample  $x_i$ , the tokenized sequence is represented as:

$$s_i = \text{Tokenizer}(x_i) = ([CLS], t_1, t_2, \dots, t_n, [SEP]), \quad (5)$$

where  $t_j$  denotes the  $j$ th subword token,  $n$  is the number of valid tokens, and  $[CLS]$  and  $[SEP]$  are special tokens inserted at the start and end of the sequence, respectively. To ensure compatibility with the student's input constraints,  $s_i$  is padded with  $[PAD]$  tokens if their length is less than 512, or truncated if they exceed this limit.

After tokenization,  $s_i$  is mapped to high-dimensional vectors by the CodeBERT embedding layer, which combines token and positional information for each token. The resulting vector representation is denoted as  $z_i$ , is fed into subsequent neural networks.

##### 4.3. Stage 1: multi-teacher feature learning

To mitigate the impact of the long-tailed distribution, VulATMHD first adopts a hierarchical learning strategy [14], dividing the 44 vulnerability types into five subgroups based on CWE abstract types (i.e., Base, Category, Class, Variant, and Deprecated) for balanced grouping. It then constructs a five-teacher ensemble, where each teacher is responsible for intra-group classification within its domain. All teachers share a common TextCNN [26] backbone, which applies convolutional filters of varying sizes followed by max-pooling to capture local semantic patterns. The resulting feature representation is denoted as  $h_i$ . Each teacher is equipped with an independent classification head consisting of a single fully connected layer that maps  $h_i$  to an output distribution over vulnerability types. For each vulnerability sample, the ensemble selects the teacher corresponding to its subgroup and takes its output distribution as the prediction.

Although contrastive learning has proven effective in the SVD task [16,17] by enlarging the feature distance between vulnerable and non-vulnerable samples, the SVC scenario investigated in this study is considerably more challenging. It involves classifying 44 vulnerability types, where the large number of classes and the highly imbalanced distribution jointly lead to an increased presence of outliers. Such



outliers introduce noisy feature distances in the feature space [18] during the early training phase, which limits the effectiveness of prior distance-based feature learning strategies.

Therefore, VulATMHD introduces adaptive triplet mining that leverages CWE abstract types as semantic priors. This is effective because CWE abstract types group related vulnerabilities while separating dissimilar ones, providing a stable prior for feature learning. It prioritizes the discrimination of semantically similar vulnerability types within the same domain, thereby mitigating interference from cross-domain outliers during feature learning. Specifically, all feature representations within a mini-batch are first normalized to ensure numerical stability:

$$\hat{\mathbf{h}}_i = \frac{\mathbf{h}_i}{\|\mathbf{h}_i\|_2}. \quad (6)$$

The distance matrix  $\mathbf{D}$  is defined to encode the cosine-based feature dissimilarities between all feature representations in the batch, where each entry  $D_{ij}$  quantifies the distance between features  $i$  and  $j$  as:

$$D_{ij} = 1 - \hat{\mathbf{h}}_i^\top \hat{\mathbf{h}}_j. \quad (7)$$

Let  $\mathbf{h}_i$  denote the anchor with CWE abstract type  $t_i$ , where  $\mathcal{P}(i)$  and  $\mathcal{N}(i)$  represent its corresponding positive and negative sample sets, respectively. VulATMHD restricts  $\mathcal{N}(i)$  to candidates sharing the same CWE abstract type:

$$\mathcal{N}_t(i) = \{j \in \mathcal{N}(i) \mid t_j = t_i\}. \quad (8)$$

When  $\mathcal{N}_t(i) \neq \emptyset$ , the hardest negative  $\mathbf{h}_{n^*}$  is the element of  $\mathcal{N}_t(i)$  with the minimum feature distance to  $\mathbf{h}_i$ ; otherwise, the same criterion is applied to  $\mathcal{N}(i)$  to select  $\mathbf{h}_{n^*}$ . Formally,  $\mathbf{h}_{n^*}$  is defined as:

$$\mathbf{h}_{n^*} = \begin{cases} \arg \min_{j \in \mathcal{N}_t(i)} D_{ij}, & \text{if } \mathcal{N}_t(i) \neq \emptyset \\ \arg \min_{j \in \mathcal{N}(i)} D_{ij}, & \text{otherwise.} \end{cases} \quad (9)$$

Based on Eq. (1), the hardest positive sample  $\mathbf{h}_{p^*}$  is selected from  $\mathcal{P}(i)$  as the farthest from the anchor  $\mathbf{h}_i$  in the feature space. Fig. 4 illustrates the selection of  $\mathbf{h}_{p^*}$  and  $\mathbf{h}_{n^*}$  in a toy three-class example. Then, according to Eq. (2), the  $\mathcal{L}_{\text{trip}}$  for  $\mathbf{h}_i$  is computed using the corresponding  $\mathbf{h}_{p^*}$  and  $\mathbf{h}_{n^*}$ .

The feature discrimination loss is computed as the mean value of  $\mathcal{L}_{\text{trip}}$  over all anchors in the batch:

$$\mathcal{L}_{\text{disc}} = \frac{1}{|\mathcal{A}|} \sum_{i \in \mathcal{A}} \mathcal{L}_{\text{trip}}(\mathbf{h}_i) \quad (10)$$

where  $\mathcal{A}$  denotes the set of anchors. This loss measures the model's ability to discriminate between feature representations of different types. It is subsequently integrated into the overall training objective of the teacher ensemble by combining it with the cross-entropy loss  $\mathcal{L}_{\text{ce}}$  [35], as follows:

$$\mathcal{L}_{\text{teacher}} = \mathcal{L}_{\text{ce}} + \lambda \cdot \mathcal{L}_{\text{disc}} \quad (11)$$

where  $\lambda$  is a hyperparameter controlling the weight of the  $\mathcal{L}_{\text{disc}}$ , and  $\mathcal{L}_{\text{ce}}$  computes the standard cross-entropy between predictions and ground-truth labels. The detailed procedure for computing  $\mathcal{L}_{\text{teacher}}$  over a mini-batch is outlined in Algorithm 1. By minimizing  $\mathcal{L}_{\text{teacher}}$  via back-propagation, the shared TextCNN backbone and the corresponding teacher-specific classification heads are updated synchronously.

By this stage, VulATMHD constructs a teacher ensemble that effectively captures fine-grained semantic relationships among vulnerability types. However, each teacher is tailored to excel at group-specific classification. Therefore, Stage 2 introduces a hybrid distillation strategy to transfer the rich knowledge of the teacher ensemble into a pre-trained student. During this process, the trained teacher ensemble is frozen and serve as knowledge sources to guide student learning.

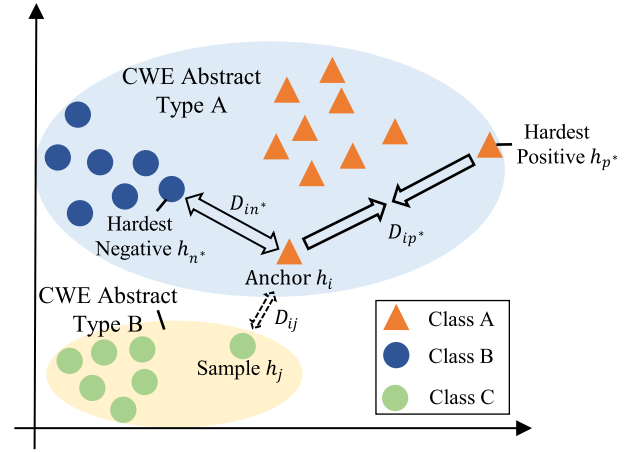


Fig. 4. An example of hardest positive and negative sample selection under adaptive triplet mining. The sample  $\mathbf{h}_{n^*}$  is selected as the hardest negative from class B within the same domain, even if a closer sample  $\mathbf{h}_j$  exists in a different domain, to reduce the impact of noisy distance signals.

---

#### Algorithm 1 Mini-batch loss calculation for the teacher

---

**Require:** Mini-batch  $B = \{(z_i, y_i, t_i)\}_{i=1}^B$ , margin  $m$ , weight  $\lambda$

**Ensure:** Mini-batch training loss  $\mathcal{L}_{\text{teacher}}$

- 1: Extract and normalize features:  $\hat{\mathbf{h}}_i = \frac{\text{TextCNN}(z_i)}{\|\text{TextCNN}(z_i)\|_2}$  for all  $i$
  - 2: Initialize  $\mathcal{A} \leftarrow \emptyset$
  - 3: **for**  $i = 1$  to  $B$  **do**
  - 4:    $\mathcal{P}(i) \leftarrow \{p \mid y_p = y_i, p \neq i\}$ ,    $\mathcal{N}(i) \leftarrow \{n \mid y_n \neq y_i\}$
  - 5:    $\mathcal{N}_t(i) \leftarrow \{n \in \mathcal{N}(i) \mid t_n = t_i\}$
  - 6:   **if**  $\mathcal{P}(i) \neq \emptyset$  and  $\mathcal{N}_t(i) \neq \emptyset$  **then**
  - 7:     Compute  $D_{ij} = 1 - \hat{\mathbf{h}}_i^\top \hat{\mathbf{h}}_j$  for  $j \in \mathcal{P}(i) \cup \mathcal{N}_t(i)$
  - 8:      $\mathbf{h}_{p^*} \leftarrow \arg \max_{p \in \mathcal{P}(i)} D_{ip}$
  - 9:      $\mathbf{h}_{n^*} \leftarrow \arg \min_{n \in \mathcal{N}_t(i)} D_{in}$  if  $\mathcal{N}_t(i) \neq \emptyset$  else  $\arg \min_{n \in \mathcal{N}(i)} D_{in}$
  - 10:      $\mathcal{L}_{\text{trip}}(\mathbf{h}_i) = \max\{D_{ip^*} - D_{in^*} + m, 0\}$
  - 11:     Add  $i$  to anchor set:  $\mathcal{A} \leftarrow \mathcal{A} \cup \{i\}$
  - 12:   **end if**
  - 13: **end for**
  - 14:  $\mathcal{L}_{\text{disc}} \leftarrow \frac{1}{|\mathcal{A}|} \sum_{i \in \mathcal{A}} \mathcal{L}_{\text{trip}}(\mathbf{h}_i)$
  - 15:  $\mathcal{L}_{\text{teacher}} \leftarrow \mathcal{L}_{\text{ce}} + \lambda \cdot \mathcal{L}_{\text{disc}}$
  - 16: **return**  $\mathcal{L}_{\text{teacher}}$
- 

#### 4.4. Stage 2: hybrid knowledge distillation

Stage 1 learns discriminative feature representations. To effectively transfer this knowledge, Stage 2 employs a hybrid distillation strategy that combines two components: (1) representation distillation, which aligns the student's feature representations with those of the teachers; and (2) label distillation, which mimics the teachers' output distributions.

Specifically, an additional token [DIS] is inserted before [SEP] in the sequence  $s_i$ , serving as a target for the student to approximate the teacher's output behavior. The modified sequence is then embedded and fed into the pre-trained CodeBERT. CodeBERT consists of 12 layers of transformer-based encoders, each comprising a multi-head self-attention mechanism followed by two feed-forward neural networks [36]. Given  $\mathbf{z}_i$ , the 12-th layer of CodeBERT outputs the learned representation  $\mathbf{h}_i^{12}$ .

To transfer the discriminative knowledge learned by the teacher, VulATMHD first performs alignment of feature dimensions. If the dimensionality of the student's representation differs from that of the teacher, a learnable linear transformation  $\mathbf{W}$  is applied to project the

student features into the teacher's feature space:

$$\tilde{\mathbf{h}}_i^{12} = \mathbf{W} \mathbf{h}_i^{12}, \quad \mathbf{W} \in \mathbb{R}^{d_t \times d_s}, \quad (12)$$

where  $\tilde{\mathbf{h}}_i^{12}$  is the transformed student representation, and  $d_s$  and  $d_t$  denote the feature dimensions of the student and teacher, respectively. For numerical stability, all student representations  $\tilde{\mathbf{h}}_i^{12}$  and teacher representations  $\mathbf{h}_j$  in the mini-batch are normalized according to Eq. (6), yielding  $\hat{\mathbf{h}}_i^{12}$  and  $\hat{\mathbf{h}}_j$ . A similarity matrix is then computed using cosine similarity across all student-teacher pairs within the mini-batch:

$$S_{ij} = (\hat{\mathbf{h}}_i^{12})^\top \hat{\mathbf{h}}_j \quad (13)$$

where  $S_{ij}$  denotes the cosine similarity between student representation  $\hat{\mathbf{h}}_i^{12}$  and teacher representation  $\hat{\mathbf{h}}_j$ . VulATMHD adopts a contrastive learning strategy [37] to align the representations of the teacher and student. For each  $\hat{\mathbf{h}}_i^{12}$ , the objective is to maximize its similarity with the corresponding teacher representation  $\hat{\mathbf{h}}_i$ , while minimizing its similarity with all other teacher representations  $\hat{\mathbf{h}}_j$  in the mini-batch, where  $j \neq i$ . Accordingly, the representation alignment loss is given by:

$$\mathcal{L}_{\text{rep}} = -\log \frac{\exp(S_{ii}/\tau)}{\sum_{j=1}^B \exp(S_{ij}/\tau)} = -\log \frac{\exp((\hat{\mathbf{h}}_i^{12})^\top \hat{\mathbf{h}}_i/\tau)}{\sum_{j=1}^B \exp((\hat{\mathbf{h}}_i^{12})^\top \hat{\mathbf{h}}_j/\tau)}, \quad (14)$$

where  $\tau$  is a temperature parameter that controls the sharpness of the similarity distribution and  $B$  denotes the number of samples in the mini-batch.

VulATMHD employs two independent Multi-Layer Perceptron (MLP) [38] classifiers: an alignment head  $\text{MLP}_{\text{align}}$  to mimic the teacher's output distribution, and a classification head  $\text{MLP}_{\text{cls}}$  for the classification task. The  $\text{MLP}_{\text{align}}$  takes the hidden state of the [DIS] token as input, and the KL divergence between the student and teacher output distributions, as defined in Eq. (3), is adopted as the label distillation loss  $\mathcal{L}_{\text{label}}$ .

Meanwhile, the  $\text{MLP}_{\text{cls}}$  performs the classification task by computing the cross-entropy loss  $\mathcal{L}_{\text{ce}}$  between the predicted distribution and the ground-truth labels. Finally, VulATMHD integrates the aforementioned loss components via a weighted summation to formulate the training objective for the student:

$$\mathcal{L}_{\text{student}} = \mathcal{L}_{\text{ce}} + \mathcal{L}_{\text{rep}} + \lambda_1 \cdot \mathcal{L}_{\text{label}}, \quad (15)$$

where  $\lambda_1$  is a hyperparameter that controls the contribution of  $\mathcal{L}_{\text{label}}$ . The overall procedure for computing  $\mathcal{L}_{\text{student}}$  over a mini-batch is summarized in Algorithm 2.

---

#### Algorithm 2 Mini-batch loss calculation for the student

---

**Require:** Mini-batch  $B = \{(z_i, y_i)\}_{i=1}^B$ , teacher features  $\{\mathbf{h}_i\}_{i=1}^B$ , teacher predictions  $\{y_i^t\}_{i=1}^B$ , weight  $\lambda_1$

**Ensure:** Mini-batch training loss  $\mathcal{L}_{\text{student}}$

```

1: Initialize  $\mathbf{W}$ ,  $\text{MLP}_{\text{align}}$ ,  $\text{MLP}_{\text{cls}}$ 
2: for  $i = 1$  to  $B$  do
3:    $\mathbf{h}_i^{12} \leftarrow \text{Pre-trained model}(z_i)$ 
4:    $\tilde{\mathbf{h}}_i^{12} \leftarrow \begin{cases} \mathbf{W} \mathbf{h}_i^{12}, & \text{if } \dim(\mathbf{h}_i^{12}) \neq \dim(\mathbf{h}_i) \\ \mathbf{h}_i^{12}, & \text{otherwise} \end{cases}$ 
5:    $\hat{\mathbf{h}}_i^{12} \leftarrow \tilde{\mathbf{h}}_i^{12}$ ,  $\hat{\mathbf{h}}_i \leftarrow \mathbf{h}_i$  (normalized via Eq. (6))
6:   Compute similarity  $S_{ij} = \text{sim}(\hat{\mathbf{h}}_i^{12}, \hat{\mathbf{h}}_j)$  for all  $j$ 
7:   Compute  $\mathcal{L}_{\text{rep}}$  from  $S_{ij}$  using Eq. (14)
8:   Compute  $\mathcal{L}_{\text{label}} = \text{KL}(y_i^t \parallel \text{MLP}_{\text{align}}(\mathbf{h}_i^{12}))$ 
9:   Compute  $\mathcal{L}_{\text{ce}} = \text{Cross-Entropy}(\text{MLP}_{\text{cls}}(\mathbf{h}_i^{12}), y_i)$ 
10: end for
11: Compute  $\mathcal{L}_{\text{student}} \leftarrow \frac{1}{B} \sum_{i=1}^B (\mathcal{L}_{\text{ce}} + \mathcal{L}_{\text{rep}} + \lambda_1 \cdot \mathcal{L}_{\text{label}})$ 
12: return  $\mathcal{L}_{\text{student}}$ 
```

---

After completing the hybrid knowledge distillation, only the fine-tuned student (CodeBERT with  $\text{MLP}_{\text{cls}}$ ) is deployed for inference. Given a new sample, it is tokenized into a numerical sequence, processed through the fine-tuned CodeBERT to extract feature representations, and classified by  $\text{MLP}_{\text{cls}}$  to determine the vulnerability type.

## 5. Experimental setup

This section introduces the Research Questions (RQs), experimental datasets, implementation settings, baseline methods, and evaluation metrics.

### 5.1. Research questions

This study defines the following five RQs to evaluate VulATMHD in terms of effectiveness, robustness, and scalability:

#### RQ1: What is the performance of VulATMHD in type-aware SVC?

**Motivation:** VulATMHD addresses the limitations of existing SVC methods in capturing fine-grained semantics and transferring knowledge. To assess its effectiveness, RQ1 compares the performance of VulATMHD against baseline methods based on Accuracy and weighted F1-score, and further reports classification results within the five CWE abstract type groups.

#### RQ2: Is the proposed adaptive triplet mining effective in enhancing feature learning in VulATMHD?

**Motivation:** VulATMHD introduces adaptive triplet mining guided by CWE abstract types as prior knowledge. RQ2 evaluates its effectiveness by comparing it with three sampling strategies: random [18], semi-hard [39], and batch-hard [21].

#### RQ3: Does each component of VulATMHD make a positive contribution to the overall performance?

**Motivation:** VulATMHD introduces adaptive triplet mining and hybrid distillation to overcome the limitations of existing SVC methods. To assess the effectiveness of these components, RQ3 conducts an ablation study to evaluate the contribution of each component.

#### RQ4: What is the robustness of VulATMHD under adversarial attacks?

**Motivation:** Recent studies [23,24,40] have shown that pre-trained models, despite their strong performance, remain susceptible to adversarial attacks. Attackers can induce incorrect predictions by making subtle perturbations that preserve semantic correctness. Therefore, RQ4 evaluates the robustness of VulATMHD and baseline methods under two adversarial attack scenarios.

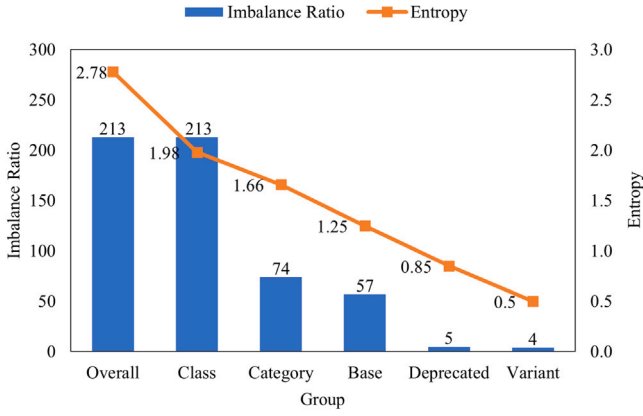
#### RQ5: Can VulATMHD be extended to the SVD task?

**Motivation:** While the primary focus of this study is on the SVC task, certain real-world applications can be sufficiently addressed by SVD alone. Therefore, RQ5 evaluates the effectiveness of VulATMHD on the SVD task by comparing it with representative SVD methods.

### 5.2. Experimental datasets

**BigVul.** This study uses the BigVul dataset, widely adopted in recent SVC research [14,15], to evaluate VulATMHD and baseline methods. Constructed by Fan et al. [22] through crawling the common vulnerabilities and exposures database, BigVul contains 8636 C/C++ vulnerability samples from over 300 software projects, covering 44 distinct vulnerability types, each identified by a CWE-ID. Each type is further annotated with its corresponding CWE abstract type, allowing for a division into five groups: Base, Category, Class, Variant, and Deprecated. Fig. 5 presents the distributional statistics of BigVul using imbalance ratio and entropy, where the former quantifies label imbalance as the ratio between the majority and minority classes, and the latter measures class uncertainty based on information entropy. When used for the SVD task, the full BigVul corpus is employed, comprising 188,636 samples, with 5.7% labeled as vulnerable.

**Devign and Reveal.** Beyond BigVul, this study further evaluates VulATMHD on the SVD task using the Devign and Reveal datasets. The Devign dataset [9] comprises 27,318 code samples collected from security-related GitHub commits in two large-scale projects, QEMU and Ffmpeg. It includes 12,436 vulnerable and 14,882 non-vulnerable



**Fig. 5.** Imbalance statistics of ungrouped and grouped vulnerability type distributions. A lower Imbalance Ratio indicates a more balanced class distribution, whereas a higher Entropy suggests greater label diversity.

samples. The Reveal dataset [10] is constructed from two widely used open-source projects, Chromium and Debian, and contains 22,734 code samples, among which only 2240 are labeled as vulnerable.

### 5.3. Baseline methods

To evaluate the effectiveness of VulATMHD on the SVC task, this study performs a comparative evaluation against the following baseline methods:

- **CodeBERT:** CodeBERT [7] is a pre-trained model designed for source code understanding. It is trained on six widely used programming languages (Python, Java, JavaScript, PHP, Ruby, and Go) with two training objectives: masked language modeling [41] and replaced token detection [42].
- **GraphCodeBERT:** GraphCodeBERT [43] is a variant of CodeBERT, designed to capture both sequential and structural information of source code. It incorporates both code token sequences and data flow graphs, leveraging masked attention to capture their interactions.
- **CodeT5+:** CodeT5+ [44] employs an encoder-decoder architecture. Compared with CodeBERT, it introduces four pre-training objectives: span denoising [45], causal language modeling [46], contrastive learning [8], and matching tasks [47].
- **Devign:** Devign [9] first transforms a code sample into a multi-edged graph, then applies GGNNs to update the node representations.
- **ReGVD:** ReGVD [25] converts code tokens into graphs, and uses a GCN with residual connections to extract a graph-level representation for prediction.
- **VulExplainer:** VulExplainer [14] adopts a hierarchical learning strategy to address the long-tailed label distribution and leverages label distillation to transfer knowledge into a pre-trained model. It is compatible with CodeBERT [7], GraphCodeBERT [43], and CodeGPT [48]. This study adopts the GraphCodeBERT variant, which achieves the highest weighted F1-score.
- **LIVABLE:** LIVABLE [15] introduces an adaptive re-weighting mechanism to balance the learning process across different CWE-IDs. This mechanism adjusts class-wise weights based on sample frequency and training dynamics.

For methods originally designed for SVD (CodeBERT, GraphCodeBERT, CodeT5+, Devign, ReGVD), this study adapts them to the SVC task by replacing their binary classification layers with an MLP classifier that outputs probability distributions over all CWE-IDs. For VulExplainer and LIVABLE, which are designed for SVC, this study applies them following the configurations detailed in their papers.

To further evaluate the scalability of VulATMHD on the SVD task, this study compares it with GraphCodeBERT, CodeT5+, Devign, ReGVD, and two representative SVD methods:

- **LineVul:** LineVul [5] leverages CodeBERT to extract semantic representations of source code for vulnerability detection.
- **PM2-CNN:** PM2-CNN [6] leverages UniXcoder to extract semantic features from source code and employs a multi-channel CNN for vulnerability detection.

### 5.4. Evaluation metrics

The predictions of a classification model can be divided into four types: (1) True Positive (TP): the number of samples correctly predicted as the positive class; (2) False Positive (FP): the number of samples incorrectly predicted as the positive class but actually belonging to the negative class; (3) True Negative (TN): the number of samples correctly predicted as the negative class; and (4) False Negative (FN): the number of samples incorrectly predicted as the negative class but actually belonging to the positive class.

This study uses Accuracy and weighted F1-score as evaluation metrics to assess the effectiveness of VulATMHD. Accuracy measures the proportion of correctly predicted samples among all samples, and is defined as follows:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}. \quad (16)$$

The weighted F1-score is computed as the average F1-score across all classes, weighted by the number of true instances in each class. This metric provides a comprehensive evaluation of model performance under long-tailed distributions. The F1-score is the harmonic mean of Precision and Recall, and is defined as follows:

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (17)$$

where precision and recall are computed as:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}. \quad (18)$$

The weighted F1-score is defined as:

$$\text{Weighted F1-score} = \sum_{c=1}^C \frac{n_c}{N} \cdot \text{F1-score}_c, \quad (19)$$

where  $C$  is the number of classes,  $N$  is the total number of instances,  $n_c$  is the number of true instances in class  $c$ , and  $\text{F1-score}_c$  is the F1-score for class  $c$ .

### 5.5. Experimental settings

The experiments are conducted on a Linux server equipped with two AMD EPYC 7663 CPUs, 512 GB of RAM, and eight AMD Instinct MI210 GPUs, each with 64 GB of VRAM. The implementation is built upon Python 3.11 and primarily relies on two DL libraries: PyTorch (2.1.2+rocm5.6) and Transformers (4.35.2).

The experimental dataset is split into training, validation, and test sets in an 8:1:1 ratio. All experiments for each RQ are conducted three times, with the mean value reported as the final result. In the teacher ensemble, the TextCNN backbone employs convolutional kernels with window sizes of 3, 4, and 5, each with 100 filters. The remaining hyperparameters for both the teacher and student are summarized in Table 1. This study evaluates VulATMHD using three pre-trained models: CodeBERT, GraphCodeBERT, and CodeT5+, whose architectures and parameter scales are summarized in Table 2. For CodeT5+, which has an encoder-decoder architecture, only the encoder component is utilized for feature extraction, with the decoder discarded to maintain consistency with the framework described in Section 4.4.

**Table 1**  
Hyperparameter settings for the teacher and student models.

| Parameter                     | Teacher | Student |
|-------------------------------|---------|---------|
| Epoch                         | 50      | 50      |
| Batch Size                    | 128     | 8       |
| Optimizer                     | Adam    | Adam    |
| Learning Rate                 | 5e-3    | 2e-5    |
| Scheduler                     | Linear  | Linear  |
| Margin $m$ (Eq. (2))          | 0.3     | –       |
| Weight $\lambda$ (Eq. (11))   | 0.4     | –       |
| Temperature $\tau$ (Eq. (14)) | –       | 0.1     |
| Weight $\lambda_1$ (Eq. (15)) | –       | 0.5     |

## 6. Experimental results and analysis

### 6.1. Answer to RQ1: What is the performance of VulATMHD in type-aware SVC?

To answer RQ1, VulATMHD is evaluated on the BigVul dataset, reporting overall Accuracy, weighted F1-score, and Accuracy for each of the five CWE abstract type groups. To further assess its representational capacity, pre-classifier features from GraphCodeBERT, VulExplainer, and VulATMHD<sub>GraphCodeBERT</sub> are compared using intra-class average distance (where lower values indicate greater intra-class compactness) and inter-class centroid distance (where higher values indicate clearer inter-class separation) [17].

Table 3 presents a performance comparison between VulATMHD and seven baseline methods. In terms of overall performance, VulATMHD achieves the superior results when integrated with CodeT5+, outperforming all baseline methods by 4.7%–29.9% in Accuracy and 5.7%–34.1% in weighted F1-score. Notably, graph-based SVC methods underperform compared to pre-trained model-based methods, with Deign and ReGVD ranking the lowest and second-lowest, respectively. Even the recently proposed SVC method, LIVABLE, only outperforms the direct use of CodeBERT and GraphCodeBERT. This observation is consistent with previous findings [5,6], and is primarily attributed to the powerful representational capacity of pre-trained models. VulExplainer alleviates the impact of long-tail distribution through a hierarchical learning strategy, leading to improved Accuracy (60.1%→63.5%) and weighted F1-score (59.7%→63.3%). However, due to its neglect of fine-grained semantic relationships among vulnerability types and insufficient knowledge transfer, the performance gains of VulExplainer remain less pronounced than those of VulATMHD. In contrast, VulATMHD further enhances the model’s discriminative capability, achieving an Accuracy of 66.4% and a weighted F1-score of 65.8%. Moreover, VulATMHD significantly improves the Accuracy (60.7%→66.2%) and weighted F1-score (60.2%→65.9%) of CodeBERT. Similarly, when built upon CodeT5+, VulATMHD also yields encouraging improvements, achieving Accuracy (64.4%→67.4%) and weighted F1-score (63.1%→66.9%).

In terms of subgroup performance, VulATMHD achieves either the best or second-best results across the first four groups, significantly outperforming all baselines. However, its performance in the Deprecated group is slightly inferior to that of LIVABLE. This underperformance is mainly attributed to the joint effect of sample size and vulnerability characteristics, as detail in Section 7.2. Nevertheless, VulATMHD retains the baseline performance of all three pre-trained models in this group and achieves further improvement on CodeT5+. Moreover, in the first four groups, VulATMHD consistently yields significant gains across all three pre-trained models.

Table 4 compares the pre-classifier feature distributions across three methods. VulATMHD<sub>GraphCodeBERT</sub> achieves superior performance by reducing the intra-class average distance by 14.2% and 11.1%, and increasing the inter-class centroid distance by 19.5% and 16.1%, compared to GraphCodeBERT and VulExplainer, respectively. These results

suggest that the learned representation of VulExplainer remains suboptimal due to its neglect of fine-grained semantic relationships among vulnerability types. In contrast, VulATMHD enhances inter-class separability and intra-class compactness through adaptive triplet mining and hybrid distillation strategy.

**Finding 1:** VulATMHD outperforms baseline methods in both overall and subgroup classification performance, and consistently achieves improvements when integrated with pre-trained models of different architectures.

### 6.2. Answer to RQ2: Is the proposed adaptive triplet mining effective in enhancing feature learning in vulatmhd?

VulATMHD introduces adaptive triplet mining in Stage 1 to enhance feature discrimination. To answer RQ2, this component is substituted with each of the following three triplet mining strategies:

- **Random triplet mining:** For each anchor in the batch, one positive and one negative sample are randomly selected to form a triplet [18].
- **Semi-hard triplet mining:** For each anchor, the nearest positive sample is selected, and a negative is chosen such that its distance to the anchor is greater than that of the positive but within a predefined margin [39].
- **Batch-hard triplet mining:** For each anchor, the hardest positive (i.e., the farthest in the feature space) and the hardest negative (i.e., the closest) are selected to form a triplet [21].

Tables 5 and 6 report the Accuracy and weighted F1-score of VulATMHD using different triplet mining strategies in Stage 1, respectively. Adaptive triplet mining yields the best performance across all three pre-trained models, with the most significant gains observed on CodeBERT, yielding a 3.9%–5.9% gain in Accuracy and a 4.3%–6.3% gain in weighted F1-score. Random triplet mining performs the worst, as its purely stochastic sampling fails to select informative sample pairs, thereby limiting feature discrimination. In contrast, semi-hard and batch-hard triplet mining leverage distance-based sampling to construct more meaningful triplets. However, both rely solely on feature distances, which can be noisy during the early training phase when representations are unstable. This often results in suboptimal triplet construction. Instead, adaptive triplet mining integrates CWE abstract types as semantic priors, enabling more semantically aligned pair selection and thus achieving superior performance.

Although several triplet mining strategies have been proposed in computer vision, such as introducing multi-similarity constraints [49] or improved margin strategies [50], they still rely on feature distances for sample selection and remain sensitive to noisy representations. Moreover, they also introduce additional hyperparameters, increasing the computational cost of training. Hence, they are excluded from this comparison.

**Finding 2:** Compared to previous triplet mining strategies, the proposed adaptive triplet mining leverages domain-specific knowledge to improve feature learning, thereby achieving superior performance.

### 6.3. Answer to RQ3: Does each component of VulATMHD make a positive contribution to the overall performance?

To answer RQ3, an ablation study is designed by removing each component of VulATMHD, resulting in the following variants:

- **w/o Adaptive Triplet Mining (ATM):** Removes adaptive triplet mining in Stage 1.
- **w/o Representation Distillation (RD):** Removes representation distillation in Stage 2, allowing the student to learn only through label distillation from the teacher ensemble.



**Table 2**  
Architectures and parameter scales of the evaluated pre-trained models.

| Model              | Type     | Architecture    | Parameter Scale |
|--------------------|----------|-----------------|-----------------|
| CodeBERT [7]       | Sequence | Encoder-only    | 125M            |
| GraphCodeBERT [43] | Graph    | Encoder-only    | 125M            |
| CodeT5+[44]        | Sequence | Encoder-decoder | 220M            |

**Table 3**  
Comparison of the performance (%) between VulATMHD and baseline methods. Results are reported for each group and overall. The best performance is highlighted in **bold**, and the second-best is underlined.

| Method                            | Group By CWE abstract types (Accuracy) |      |          |         |            | Accuracy | weighted F1-score |
|-----------------------------------|----------------------------------------|------|----------|---------|------------|----------|-------------------|
|                                   | Class                                  | Base | Category | Variant | Deprecated |          |                   |
| Devign                            | 59.2                                   | 42.3 | 45.8     | 41.5    | 34.6       | 51.9     | 49.9              |
| ReGVD                             | 64.0                                   | 53.0 | 57.5     | 43.9    | 57.7       | 59.5     | 58.5              |
| CodeBERT                          | 61.7                                   | 61.3 | 62.1     | 46.3    | 53.9       | 60.7     | 60.2              |
| CodeT5+                           | 67.6                                   | 61.3 | 66.7     | 41.5    | 50.0       | 64.4     | 63.1              |
| GraphCodeBERT                     | 62.5                                   | 60.7 | 58.8     | 36.6    | 57.7       | 60.1     | 59.7              |
| VulExplainer                      | 65.3                                   | 64.3 | 64.7     | 41.5    | 57.7       | 63.5     | 63.3              |
| LIVABLE                           | 65.1                                   | 56.5 | 60.1     | 41.5    | 61.5       | 61.2     | 60.4              |
| VulATMHD <sub>CodeBERT</sub>      | 66.5                                   | 67.9 | 68.0     | 58.5    | 53.9       | 66.2     | 65.9              |
| VulATMHD <sub>CodeT5+</sub>       | 70.1                                   | 66.1 | 67.3     | 53.7    | 53.9       | 67.4     | 66.9              |
| VulATMHD <sub>GraphCodeBERT</sub> | 68.0                                   | 68.5 | 65.4     | 48.7    | 57.7       | 66.4     | 65.8              |

**Table 4**  
Evaluation of pre-classifier feature distribution distances for GraphCodeBERT, VulExplainer, and VulATMHD<sub>GraphCodeBERT</sub>. The best results are highlighted in **bold**.

| Method                            | Intra-class avg distance | Inter-class centroid distance |
|-----------------------------------|--------------------------|-------------------------------|
| GraphCodeBERT                     | 0.148                    | 0.272                         |
| VulExplainer                      | 0.143                    | 0.280                         |
| VulATMHD <sub>GraphCodeBERT</sub> | 0.127                    | 0.325                         |

**Table 5**  
Accuracy (%) comparison of VulATMHD under different triplet mining strategies. Best results are highlighted in **bold**.

| Method          | CodeBERT | CodeT5+ | GraphCodeBERT |
|-----------------|----------|---------|---------------|
| Random          | 62.5     | 64.4    | 63.3          |
| Semi-hard       | 63.3     | 65.3    | 63.7          |
| Batch-hard      | 63.7     | 65.4    | 64.1          |
| Adaptive (Ours) | 66.2     | 67.4    | 66.4          |

**Table 6**  
Weighted F1-score (%) comparison of VulATMHD under different triplet mining strategies. Best results are highlighted in **bold**.

| Method          | CodeBERT | CodeT5+ | GraphCodeBERT |
|-----------------|----------|---------|---------------|
| Random          | 62.0     | 63.9    | 62.9          |
| Semi-hard       | 62.9     | 64.8    | 63.1          |
| Batch-hard      | 63.2     | 65.1    | 63.7          |
| Adaptive (Ours) | 65.9     | 66.9    | 65.8          |

- **w/o Label Distillation (LD):** Removes the label distillation in Stage 2, allowing the student to learn only through representation distillation from the teacher ensemble.
- **w/o ATM&LD:** Removes both adaptive triplet mining and label distillation.
- **w/o ATM&RD:** Removes both adaptive triplet mining and representation distillation.

Note that the performance of directly using the pre-trained model (i.e., without any of the above components) has been presented in Section 6.1 and is omitted here.

Tables 7 and 8 present the Accuracy and weighted F1-score of different ablation variants. Overall, VulATMHD consistently outperforms all variants across all pre-trained models. It is observed that w/o LD leads to a slight yet consistent drop of 0.9–2.2% in Accuracy and 1.7–2.6%

**Table 7**  
Accuracy (%) comparison of different ablation variants. Best results are highlighted in **bold**.

| Method          | CodeBERT | CodeT5+ | GraphCodeBERT |
|-----------------|----------|---------|---------------|
| w/o ATM         | 63.2     | 65.7    | 64.6          |
| w/o RD          | 63.1     | 65.5    | 64.3          |
| w/o LD          | 65.6     | 66.3    | 65.0          |
| w/o ATM & LD    | 62.7     | 65.2    | 63.7          |
| w/o ATM & RD    | 62.4     | 64.6    | 63.5          |
| Full (VulATMHD) | 66.2     | 67.4    | 66.4          |

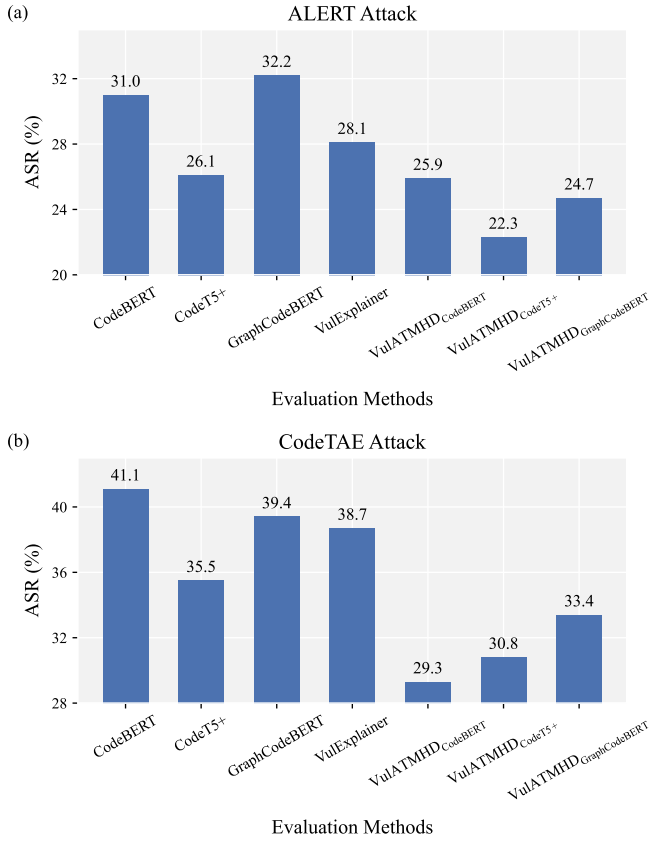
in weighted F1-score across all three models. Both w/o ATM and w/o RD exhibit larger performance declines across all models, though the reductions remain less pronounced than those observed in the w/o ATM & LD and w/o ATM & RD. This highlights the effectiveness of each component and their synergistic contribution to VulATMHD. Specifically, introducing adaptive triplet mining alone improves the discriminability of learned representations, but label distillation alone is insufficient to fully transfer the rich feature information from the teacher. Conversely, applying only hybrid distillation lacks explicit feature-level supervision, limiting further performance gains. Their integration provides complementary benefits, resulting in the optimal performance.

**Finding 3:** The ablation study confirms the effectiveness of both adaptive triplet mining and hybrid distillation in improving the performance of VulATMHD.

#### 6.4. Answer to RQ4: What is the robustness of VulATMHD under adversarial attacks?

To answer RQ4, the robustness of VulATMHD is evaluated under different adversarial attacks. Two representative adversarial attacks are considered in this study:

- **ALERT:** ALERT generates adversarial examples through variable renaming. It employs masked language modeling to suggest semantically related substitutions, and uses a combination of greedy search and genetic algorithms to select effective replacements [23].
- **CodeTAE:** CodeTAE extends ALERT by introducing additional semantics-preserving perturbations (e.g., inserting unused variables and empty loops). It employs a genetic algorithm to maximize the feature-space distance between the adversarial and



**Fig. 6.** ASR (%) of different SVC methods under adversarial attacks: (a) ALERT attack and (b) CodeTAE attack. A lower ASR indicates stronger robustness.

**Table 8**

Weighted F1-score (%) comparison of different ablation variants. Best results are highlighted in **bold**.

| Method          | CodeBERT    | CodeT5+     | GraphCodeBERT |
|-----------------|-------------|-------------|---------------|
| w/o ATM         | 62.7        | 65.0        | 64.0          |
| w/o RD          | 62.5        | 64.6        | 63.7          |
| w/o LD          | 64.8        | 65.5        | 64.3          |
| w/o ATM & LD    | 62.2        | 64.7        | 63.2          |
| w/o ATM & RD    | 61.7        | 63.6        | 63.3          |
| Full (VulATMHD) | <b>65.9</b> | <b>66.9</b> | <b>65.8</b>   |

original samples within the pre-trained encoder, thus enhancing attack efficacy [24].

The attack configuration follows that of CodeTAE: adversarial examples are generated from vulnerable samples that are correctly classified by the target SVC models. The Attack Success Rate (ASR) is used as the evaluation metric, measuring the proportion of adversarial examples that cause misclassification. Since adversarial attack studies focus on pre-trained models, the comparison is restricted to baseline methods that are also based on pre-trained architectures.

Fig. 6 presents the results under two adversarial attacks. Across both scenarios, VulATMHD substantially improves the robustness of pre-trained models. Under the ALERT attack, the improvement is most pronounced on GraphCodeBERT, where the ASR drops from 32.2% to 24.7%. In contrast, VulExplainer only reduces the ASR to 28.1%. Under the CodeTAE attack, the most significant improvement is observed on CodeBERT, with the ASR decreasing from 41.1% to 29.3%. Meanwhile, on GraphCodeBERT, VulATMHD reduces the ASR from 39.4% to 33.4%, while VulExplainer still exhibits a higher ASR of 38.7%.

Experimental results demonstrate that VulATMHD consistently outperforms both the original pre-trained models and VulExplainer under

**Table 9**

Performance (%) of VulATMHD' and baseline methods on the Devign dataset. The best performance is highlighted in **bold**, and the second-best is underlined.

| Method                             | Accuracy    | F1-score    |
|------------------------------------|-------------|-------------|
| LineVul                            | 62.8        | 51.5        |
| CodeT5+                            | 62.9        | 55.4        |
| GraphCodeBERT                      | 63.3        | 56.5        |
| Devign                             | 58.4        | 54.9        |
| ReGVD                              | 56.4        | 51.1        |
| PM2-CNN                            | 63.2        | 53.2        |
| VulATMHD' <sub>CodeBERT</sub>      | 64.6        | 54.4        |
| VulATMHD' <sub>CodeT5+</sub>       | <b>65.1</b> | <b>58.2</b> |
| VulATMHD' <sub>GraphCodeBERT</sub> | <u>64.9</u> | <u>57.0</u> |

**Table 10**

Performance (%) of VulATMHD' and baseline methods on the Reveal dataset. The best performance is highlighted in **bold**, and the second-best is underlined.

| Method                             | Accuracy    | F1-score    |
|------------------------------------|-------------|-------------|
| LineVul                            | 92.3        | 35.6        |
| CodeT5+                            | 91.7        | 32.3        |
| GraphCodeBERT                      | 91.8        | 35.4        |
| Devign                             | 88.7        | 34.1        |
| ReGVD                              | 92.1        | 28.0        |
| PM2-CNN                            | 90.3        | 37.4        |
| VulATMHD' <sub>CodeBERT</sub>      | 91.9        | 41.0        |
| VulATMHD' <sub>CodeT5+</sub>       | <u>92.5</u> | <u>43.1</u> |
| VulATMHD' <sub>GraphCodeBERT</sub> | <b>92.6</b> | <b>44.2</b> |

adversarial attacks. This robustness stems from its ability to capture fine-grained semantic relationships between vulnerability types and to establish more discriminative feature boundaries. In practical adversarial scenarios, attacks are typically subtle and semantics-preserving, meaning the original semantics of the input remain largely unchanged. Consequently, adversarial examples tend to stay close to their original counterparts in the feature space [24]. However, both the original pre-trained models and VulExplainer fail to fully exploit fine-grained semantic relationships during training, resulting in increased feature overlap. This makes them more vulnerable to misclassification under adversarial attacks.

**Finding 4:** VulATMHD exhibits superior robustness against adversarial attacks, achieving lower ASR than baseline methods. This robustness is attributed to its enhanced feature discriminability enabled by adaptive triplet mining.

#### 6.5. Answer to RQ5: Can VulATMHD be extended to the SVD task?

To answer RQ5, this study extends VulATMHD to the SVD task, resulting in a variant referred to as VulATMHD'. Specifically, two main modifications are introduced: (1) the original multi-teacher classification head in Stage 1 is replaced with a single binary classification head, and adaptive triplet mining is simplified to a batch-hard mining, which selects the farthest positive and the nearest negative sample; (2) the classifier in the student is also modified to produce binary outputs. Except for these changes, all other architectural components and hyperparameter settings remain consistent with the original VulATMHD.

Table 9 presents the performance of VulATMHD' and representative SVD methods on the Devign dataset. VulATMHD'<sub>CodeT5+</sub> achieves the best results, outperforming baseline methods by 2.8%–15.4% in Accuracy and 3.0%–13.9% in F1-score. It can be observed that VulATMHD' yields performance improvements across all three pre-trained models (where LineVul can be regarded as a direct implementation based on CodeBERT).

Table 10 presents the performance of VulATMHD' and representative SVD methods on the Reveal dataset. VulATMHD'<sub>GraphCodeBERT</sub>

**Table 11**

Performance (%) of VulATMHD' and baseline methods on the BigVul dataset. The best performance is highlighted in **bold**, and the second-best is underlined.

| Method                             | Accuracy    | F1-score    |
|------------------------------------|-------------|-------------|
| LineVul                            | 99.0        | 91.0        |
| CodeT5+                            | <b>99.2</b> | 92.6        |
| GraphCodeBERT                      | 95.0        | 92.3        |
| Devign                             | 92.8        | 28.6        |
| ReGVD                              | 93.5        | 32.5        |
| PM2-CNN                            | 99.1        | 93.1        |
| VulATMHD' <sub>CodeT5+</sub>       | 98.9        | 91.6        |
| VulATMHD' <sub>CodeT5+</sub>       | <b>99.2</b> | <u>93.7</u> |
| VulATMHD' <sub>GraphCodeBERT</sub> | 96.1        | <b>93.9</b> |

and VulATMHD'<sub>CodeT5+</sub> achieve the best and second-best overall performance, respectively. Notably, VulATMHD'<sub>GraphCodeBERT</sub> achieves significant gains in F1-score over all baseline methods, with improvements of 18.2%–57.9%. While VulATMHD' consistently improves F1-score across all three pre-trained models, the gains in Accuracy are modest and even decline on CodeBERT. These results can be attributed to the severe class imbalance in the Reveal dataset. VulATMHD' enhances the model's ability to distinguish minority-class samples (i.e., vulnerable samples), leading to a notable improvement in F1-score. In contrast, Accuracy, being dominated by the majority class, is less sensitive to these improvements and may slightly degrade due to this trade-off.

Table 11 presents the performance of VulATMHD' and representative SVD methods on the BigVul dataset. VulATMHD'<sub>CodeT5+</sub> and CodeT5+ achieved the highest Accuracy, while VulATMHD'<sub>CodeT5+</sub> obtained the best F1-score. Similar to the trend on Reveal, VulATMHD' shows limited Accuracy improvements, with a 1.2% gain observed only on GraphCodeBERT. In contrast, the F1-score achieves more pronounced gains across all three pre-trained models, reaching 1.7% on GraphCodeBERT. While both BigVul and Reveal are class-imbalanced, BigVul has a lower vulnerable rate (5.7% vs. 9.9%) and a much larger sample size, which enhances baseline performance but increases the difficulty of learning from minority classes. Nevertheless, VulATMHD' achieves modest performance gains while maintaining performance across all pre-trained models.

Overall, VulATMHD' is compatible with various pre-trained models and outperforms representative SVD methods. However, its gains are less pronounced compared to those achieved by VulATMHD on the SVC task. This can be attributed to two factors: (1) VulATMHD is designed for the SVC task and excels at capturing fine-grained semantic relationships between different vulnerability types, whereas SVD involves only binary classification and does not fully utilize this capability; (2) To highlight the scalability of VulATMHD, VulATMHD' adopts the same hyperparameter settings as VulATMHD without SVD-specific tuning, which may limit its effectiveness.

**Finding 5:** VulATMHD exhibits favorable scalability and can be effectively extended to the SVD task with only slight architectural changes.

## 7. Discussion

Building on the validated superiority of VulATMHD in the five RQs, this section explores the factors behind its effectiveness. We discuss two main advantages, qualitatively analyze its practical utility and limitations, evaluate the Stage 1 feature extractor and key hyperparameters, and compare it with representative long-tailed learning algorithms. All discussions are conducted based on GraphCodeBERT, where VulATMHD demonstrates the most significant improvements.

**Table 12**

Accuracy (%) of different methods on selected vulnerability type pairs. The best performance is highlighted in **bold**.

| Method        | CWE-119     | CWE-125     | CWE-22      | CWE-284     |
|---------------|-------------|-------------|-------------|-------------|
| GraphCodeBERT | 70.9        | 66.7        | 0.0         | 52.9        |
| LIVABLE       | 72.8        | 68.8        | <b>66.7</b> | 52.9        |
| VulExplainer  | 74.2        | 71.4        | 33.3        | 52.9        |
| VulATMHD      | <b>78.4</b> | <b>73.0</b> | <b>66.7</b> | <b>58.8</b> |

### 7.1. Why does VulATMHD work?

This section discusses two advantages of VulATMHD, which explain its superior effectiveness compared with baseline methods in SVC.

**(1) More effective discrimination between similar vulnerability types.** VulATMHD introduces adaptive triplet mining that leverages CWE abstract types as prior knowledge, effectively enlarging the distance between similar vulnerability types during feature learning, thereby yielding clearer decision boundaries. In contrast, baseline methods such as VulExplainer and LIVABLE, neglect the fine-grained semantic relationships between vulnerability types.

The experimental findings further confirm the effectiveness of the proposed design. Moreover, this section examines two representative pairs of vulnerability types that are similar in code patterns: CWE-119 and CWE-125 from the head classes in BigVul, and CWE-22 (Path Traversal) and CWE-284 (Improper Access Control) from the tail classes, and compares the performance of VulATMHD and the baseline methods on these pairs in Table 12. Both CWE-22 and CWE-284 involve access control issues, with the former focusing on path traversal and the latter on permission misconfiguration. The results indicate that, although VulExplainer and LIVABLE improve the classification performance, they still fall short of VulATMHD in distinguishing these two similar pairs.

**(2) More effective knowledge transfer.** VulATMHD constructs its classification framework on top of a pre-trained model, leveraging the contextual semantics learned from large-scale code corpora. During training, VulATMHD transfers knowledge from the teacher ensemble at both the feature representation and decision output levels. At the feature level, contrastive learning is employed to reduce the distributional gap between the two, thereby enhancing semantic consistency; at the label level, KL divergence is introduced to align the predicted distributions, enabling comprehensive knowledge transfer.

### 7.2. Qualitative analysis

Section 7.1 demonstrates VulATMHD's strength in discriminating semantically similar vulnerabilities using two representative CWE pairs. To gain deeper insights into VulATMHD's practical utility and limitations, this section provides a qualitative analysis of its effectiveness on the Top-25 most dangerous CWE-IDs and its underperformance against LIVABLE in the Deprecated group.

Maintained by the CWE community, the 2024 CWE Top-25 most dangerous CWE-IDs [51] ranks the most common and dangerous CWE-IDs based on the prevalence and severity of publicly reported vulnerabilities over the past two years. Therefore, performance on these CWE-IDs most directly reflects the practical significance of SVC methods and is also a primary focus for security engineers. Table 13 reports results for the Top-25 CWE-IDs observed in our dataset. From the results, VulATMHD outperforms LIVABLE on seven CWE-IDs and achieves comparable performance on three others, while outperforming VulExplainer on six CWE-IDs and matching its performance on four. VulATMHD's strong performance on these CWE-IDs stems from its effective discrimination. Notably, these common and dangerous CWE-IDs are often associated with many similar vulnerability types. For example, as noted earlier, both CWE-119 and CWE-125 rank among the Top-25 CWE-IDs. Similarly, VulATMHD excels on CWE-416 (Use

**Table 13**

Accuracy (%) of different methods on the Top-25 most dangerous CWE-IDs. The best performance is highlighted in **bold**.

| Rank | CWE-ID  | LIVABLE     | VulExplainer | VulATMHD    |
|------|---------|-------------|--------------|-------------|
| 1    | CWE-79  | 40.0        | <b>60.0</b>  | <b>60.0</b> |
| 2    | CWE-787 | <b>50.0</b> | <b>50.0</b>  | <b>50.0</b> |
| 5    | CWE-22  | <b>66.7</b> | 33.3         | <b>66.7</b> |
| 6    | CWE-125 | 68.8        | 71.4         | <b>73.0</b> |
| 8    | CWE-416 | 42.4        | 42.4         | <b>48.5</b> |
| 12   | CWE-20  | 58.3        | 62.6         | <b>64.4</b> |
| 15   | CWE-269 | <b>75.0</b> | <b>75.0</b>  | <b>75.0</b> |
| 20   | CWE-119 | 72.8        | 74.2         | <b>78.4</b> |
| 21   | CWE-476 | 61.9        | 71.4         | <b>76.2</b> |
| 23   | CWE-190 | 74.2        | <b>77.4</b>  | <b>77.4</b> |

**Table 14**

Accuracy (%) of VulATMHD and LIVABLE on four selected CWE-IDs.

| Group      | CWE-ID  | Proportion (%) | LIVABLE | VulATMHD |
|------------|---------|----------------|---------|----------|
| Deprecated | CWE-17  | 0.59           | 60.0    | 40.0     |
|            | CWE-18  | 0.39           | 100.0   | 75.0     |
| Variant    | CWE-415 | 0.94           | 37.5    | 50.0     |
|            | CWE-416 | 3.82           | 42.4    | 48.5     |

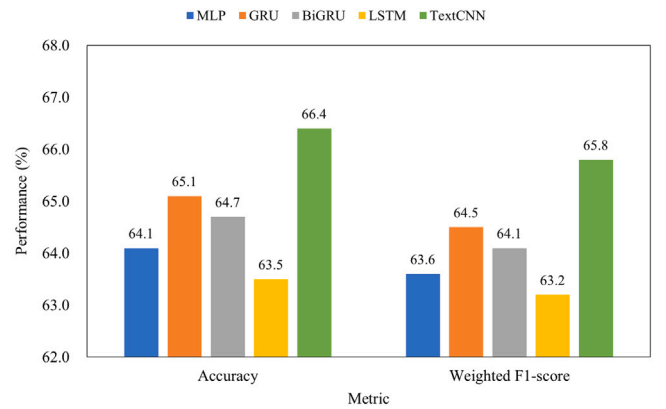
After Free); its variant CWE-415 (Double Free) is also represented in our dataset. By capturing fine-grained inter-type semantic relationships, VulATMHD achieves superior performance on these high-risk CWE-IDs, further highlighting its potential for practical deployment.

RQ1 shows that, although both the Deprecated and Variant groups have limited types and samples, VulATMHD underperforms LIVABLE in the former but remains ahead in the latter. Specifically, the Deprecated group includes three CWE-IDs, with underperformance on CWE-17 (Code) and CWE-18 (Source Code) being the driver of its shortfall. By contrast, the Variant group includes two CWE-IDs, CWE-415 and CWE-416. Table 14 reports the performance of VulATMHD and LIVABLE on these four CWE-IDs. Based on the results, the difference is primarily driven by the joint effect of sample size and vulnerability characteristics. On the one hand, CWE-17 and CWE-18 have fewer samples and belong to broad source-code-level types with limited semantic overlap, where LIVABLE's adaptive re-weighting places more effective emphasis on them. On the other hand, CWE-415 and CWE-416 focus on pointer lifetime defects and exhibit more similar code patterns, a scenario that favors VulATMHD, which excels at capturing inter-type semantic relationships. As a result, although CWE-415 accounts for only 0.94% of the samples, VulATMHD still outperforms LIVABLE. It is worth noting that CWE-17 and CWE-18 concern code quality or compiler-related issues with lower audit priority than high-risk types (e.g., CWE-416) [51]. As CWE-17/18 are retired and no longer recommended as mapping targets [52,53], their underperformance has limited practical impact on VulATMHD.

### 7.3. Effectiveness of TextCNN in stage 1

In Stage 1, TextCNN is adopted as the feature extractor to capture the local semantics of vulnerability samples. To assess its effectiveness, four additional DL models are included for comparison:

- **MLP**: A feedforward neural network composed of two fully connected layers [38].
- **Gated Recurrent Unit (GRU)**: A recurrent neural network that incorporates update and reset gates to mitigate the vanishing gradient problem and regulate information flow [54].
- **Bidirectional Gated Recurrent Unit (BiGRU)**: A bidirectional extension [55] of GRU that captures both forward and backward contextual dependencies.
- **Long Short Term Memory (LSTM)**: A recurrent neural network enhanced with input, forget, and output gates to better capture long-range dependencies within the input [56].



**Fig. 7.** Performance (%) of different DL models as feature extractors in Stage 1.

Fig. 7 presents the performance comparison of VulATMHD using five different DL models. As shown, TextCNN outperforms all other models, with improvements of 2.0%–4.6% in Accuracy and 2.0%–4.1% in weighted F1-score. Among the three recurrent models, GRU and BiGRU perform slightly worse than TextCNN, while LSTM yields the lowest performance overall. These results indicate that, compared with recurrent neural networks that emphasize long-range dependencies, the local feature extraction of TextCNN is more effective for distinguishing between vulnerability types. Given the suboptimal performance of BiGRU and LSTM, the BiLSTM model is not further considered in this study.

### 7.4. Hyperparameter analysis

The influence of the feature discrimination loss weight  $\lambda$  in Stage 1 and the label distillation loss weight  $\lambda_1$  in Stage 2 is evaluated by varying their values from 0.1 to 0.6. The results are summarized in Table 15. Additionally, the margin  $m$  of  $\mathcal{L}_{\text{disc}}$  is analyzed over the same range, and its effect is summarized in Table 16.

The feature discrimination loss weight  $\lambda$  in Stage 1 exhibits a trend of increasing performance followed by a subsequent decrease. When  $\lambda = 0.1$ , VulATMHD underperforms in both Accuracy and weighted F1-score. Performance improves as  $\lambda$  increases, peaking at  $\lambda = 0.4$ , but slightly declines at  $\lambda = 0.5$  and 0.6. These results suggest that a



**Table 15**

Performance (%) of VulATMHD under different  $\lambda$  and  $\lambda_1$  settings. The best performance is highlighted in **bold**.

| Stage 1    |             |                   | Stage 2     |             |                   |
|------------|-------------|-------------------|-------------|-------------|-------------------|
| $\lambda$  | Accuracy    | weighted F1-score | $\lambda_1$ | Accuracy    | weighted F1-score |
| 0.1        | 64.3        | 63.8              | 0.1         | 63.2        | 62.5              |
| 0.2        | 65.6        | 64.9              | 0.2         | 65.4        | 64.9              |
| 0.3        | 66.1        | 65.4              | 0.3         | 65.7        | 65.3              |
| <b>0.4</b> | <b>66.4</b> | <b>65.8</b>       | 0.4         | 66.2        | 65.7              |
| 0.5        | 65.7        | 65.2              | <b>0.5</b>  | <b>66.4</b> | <b>65.8</b>       |
| 0.6        | 64.2        | 63.5              | 0.6         | 65.9        | 65.6              |

**Table 16**

Performance (%) of VulATMHD under different  $m$  settings. The best performance is highlighted in **bold**.

| Triplet loss $\mathcal{L}_{disc}$ |             |                   |
|-----------------------------------|-------------|-------------------|
| $m$                               | Accuracy    | weighted F1-score |
| 0.1                               | 65.7        | 64.9              |
| 0.2                               | 66.1        | 65.3              |
| <b>0.3</b>                        | <b>66.4</b> | <b>65.8</b>       |
| 0.4                               | 65.5        | 64.7              |
| 0.5                               | 64.0        | 63.6              |
| 0.6                               | 64.1        | 63.3              |

moderate increase in  $\lambda$  helps VulATMHD better distinguish between vulnerability types, while an excessively large  $\lambda$  causes the discrimination loss to dominate gradient updates, ultimately hindering overall performance.

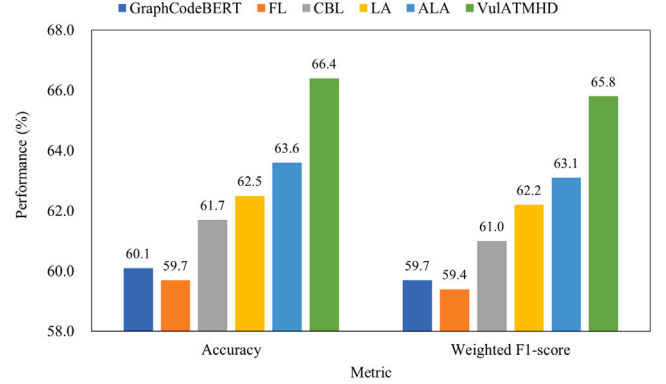
The label distillation loss weight  $\lambda_1$  in Stage 2 shows a similar trend to  $\lambda$ . When  $\lambda_1 = 0.1$ , VulATMHD achieves the lowest Accuracy and weighted F1-score. Performance improves with increasing  $\lambda_1$  and peaks at  $\lambda_1 = 0.5$ , after which it slightly declines. As performance remains stable and VulATMHD already demonstrates substantial gains over the baselines, higher values of  $\lambda_1$  are not further explored. These results indicate that a moderate increase in  $\lambda_1$  effectively facilitates knowledge transfer, while further increases offer limited additional benefit.

The margin  $m$  of  $L_{trip}$  exhibits a gradual increase followed by a decline. At  $m = 0.1$ , a small margin yields a loose separation constraint, failing to establish a sufficiently discriminative space; as  $m$  increases, the decision boundary becomes clearer, reaching optimal performance at  $m = 0.3$ . However, further enlarging  $m$  renders the constraint too restrictive, reducing the proportion of valid triplets and thereby limiting optimization. Consequently, performance declines markedly from  $m \geq 0.5$  relative to earlier settings. These results indicate that a moderate increase in  $m$  helps form a clearer inter-class decision boundary, while an overly large  $m$  imposes overly strict constraints that limit optimization.

### 7.5. Comparison with long-tailed algorithms from external domains

In computer vision, long-tailed learning algorithms alleviate class imbalance by reweighting sample losses and have achieved remarkable success in various tasks such as image classification. Since the BigVul dataset used in this study exhibits a long-tailed distribution, this section compares VulATMHD with several representative long-tailed learning algorithms from external domains. Specifically, the following four algorithms are selected to optimize the GraphCodeBERT for the SVC task:

- **Focal Loss (FL):** FL down-weights the loss of easily classified samples, encouraging the model to focus on hard and minority-class instances, thereby mitigating the impact of class imbalance [57].
- **Class-Balanced Loss (CBL):** CBL computes the effective number of samples for each class using a non-linear transformation of class frequency, and reweights the loss to strengthen learning on underrepresented classes [58].



**Fig. 8.** Performance (%) comparison between VulATMHD and external long-tailed algorithms.

- **Logit Adjustment (LA):** LA adjusts the logits before the softmax operation by subtracting the logarithm of each class's prior probability, reducing prediction bias caused by imbalance [59].
- **Adaptive Logit Adjustment (ALA):** builds on LA by introducing a dynamic shift term that accounts for both class frequency and difficulty, enabling finer-grained logit calibration [60].

Fig. 8 shows the performance comparison between VulATMHD and the aforementioned long-tailed learning algorithms. VulATMHD outperforms all competing algorithms, achieving gains of 4.4%–11.2% in Accuracy and 4.3%–10.8% in weighted F1-score. Except for FL, all long-tailed algorithms lead to performance improvements when applied to GraphCodeBERT. However, their overall effectiveness remains lower than that of VulATMHD. This is because these methods primarily focus on class reweighting and, like VulExplainer and LIVABLE, fail to capture the fine-grained semantic relationships among vulnerability types, ultimately limiting their representational capacity.

### 8. Threats to validity

One threat stems from the constructed dataset. Following VulExplainer [14] and LIVABLE [15], this study evaluated VulATMHD on a C/C++ dataset covering 44 vulnerability types, but has not been validated on other languages such as Java and Python. In theory, the ability of VulATMHD in capturing inter-class semantic relationships is transferable across programming languages, as vulnerability pairs with similar code patterns are commonly observed across languages. Future work will validate its effectiveness on diverse programming languages.

The second threat stems from generalizability across pre-trained models. VulATMHD is compatible with any pre-trained model with an encoder component, but this study only validated it on three representative models: GraphCodeBERT, CodeBERT, and CodeT5+. To mitigate this threat, models covering different architectures were selected for validation. While experimental results show significant improvements on these models, the same advantages cannot be guaranteed for all pre-trained models.

The third threat stems from the rapid development of generative Large Language Models (LLMs). In recent years, LLMs (e.g., Gemma [61], GPT [62]) have demonstrated strong potential in code-related tasks. While existing research indicates [63–65] that pre-trained models remain superior in vulnerability classification, the rapid evolution of LLMs may enable them to surpass VulATMHD in type-aware SVC.

## 9. Conclusion

This study proposes VulATMHD, a type-aware SVC framework that integrates adaptive triplet mining with hybrid distillation. By leveraging CWE abstract types as prior knowledge, VulATMHD adopts a multi-teacher architecture and introduces adaptive triplet mining to capture fine-grained relationships between types. Furthermore, a hybrid distillation strategy is introduced to facilitate effective knowledge transfer. Experimental results demonstrate that VulATMHD is compatible with various pre-trained models and achieves significant improvements over baseline methods on the BigVul dataset. In addition, VulATMHD exhibits superior robustness and scalability across downstream tasks.

Although VulATMHD achieves competitive performance, there remains considerable scope for further exploration. We plan to incorporate real-world similar-but-patched code [66] into our evaluation to further assess VulATMHD's false positive rate. In parallel, following [67], we will package VulATMHD as a Visual Studio Code extension to improve its usability in real-world development workflows.

## CRediT authorship contribution statement

**Xuanye Wang:** Writing – original draft, Visualization, Methodology, Data curation. **Lu Lu:** Writing – review & editing, Validation, Supervision, Methodology.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgments

This work was supported by the Natural Science Foundation of Guangdong Province, China (2024A1515010204) and the Technological Research Project of Southern Power Grid Company (ZBKJXM 20232483).

## Data availability

The dataset and models used in this study are publicly available at <https://github.com/wxyzjp123/VulATMHD>.

## References

- [1] Lu Wang, Chenhan Sun, Chongyang Zhang, Weikun Nie, Kaiyuan Huang, Application of knowledge graph in software engineering field: A systematic literature review, *Inf. Softw. Technol.* 164 (2023) 107327.
- [2] Wikipedia - 2024 National Public Data Breach, 2024, [https://en.wikipedia.org/wiki/2024\\_National\\_Public\\_Data\\_breach](https://en.wikipedia.org/wiki/2024_National_Public_Data_breach).
- [3] MITRE corporation - Common weakness enumeration, 2006, <https://cwe.mitre.org/>.
- [4] SecurityScorecard - CVE Details, 2024, <https://www.cvedetails.com/browse-by-date.php>.
- [5] Michael Fu, Chakkrit Tantithamthavorn, Linevul: A transformer-based line-level vulnerability prediction, in: *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 608–620.
- [6] Jingyu Liu, Jun Ai, Minyan Lu, Jie Wang, Haoxiang Shi, Semantic feature learning for software defect prediction from source code and external knowledge, *J. Syst. Softw.* 204 (2023) 111753.
- [7] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al., Codebert: A pre-trained model for programming and natural languages, 2020, arXiv preprint [arXiv:2002.08155](https://arxiv.org/abs/2002.08155).
- [8] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, Jian Yin, Unixcoder: Unified cross-modal pre-training for code representation, 2022, arXiv preprint [arXiv:2203.03850](https://arxiv.org/abs/2203.03850).
- [9] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, Yang Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, *Adv. Neural Inf. Process. Syst.* 32 (2019).
- [10] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, Baishakhi Ray, Deep learning based vulnerability detection: Are we there yet? *IEEE Trans. Softw. Eng.* 48 (9) (2021) 3280–3296.
- [11] Hong Quy Nguyen, Thong Hoang, Hoa Khanh Dam, Aditya Ghose, Graph-based explainable vulnerability prediction, *Inf. Softw. Technol.* 177 (2025) 107566.
- [12] Hong Quy Nguyen, Thong Hoang, Hoa Khanh Dam, Guoxin Su, Zhenchang Xing, Qinghua Lu, Jiamou Sun, Human-understandable explanation for software vulnerability prediction, *J. Syst. Softw.* (2025) 112455.
- [13] Qian Wang, Yuying Gao, Jiadong Ren, Bing Zhang, An automatic classification algorithm for software vulnerability based on weighted word vector and fusion neural network, *Comput. Secur.* 126 (2023) 103070.
- [14] Michael Fu, Van Nguyen, Chakkrit Kla Tantithamthavorn, Trung Le, Dinh Phung, Vulexplainer: A transformer-based hierarchical distillation for explaining vulnerability types, *IEEE Trans. Softw. Eng.* 49 (10) (2023) 4550–4565.
- [15] Xin-Cheng Wen, Cuiyun Gao, Feng Luo, Haoyu Wang, Ge Li, Qing Liao, LIVABLE: exploring long-tailed classification of software vulnerability types, *IEEE Trans. Softw. Eng.* (2024).
- [16] Rongcun Wang, Senlei Xu, Yuan Tian, Xingyu Ji, Xiaobing Sun, Shujuang Jiang, SCL-CVD: Supervised contrastive learning for code vulnerability detection via GraphCodeBERT, *Comput. Secur.* 145 (2024) 103994.
- [17] Chunyong Zhang, Liangwei Yao, Yang Xin, Vulnerability detection with graph attention network and metric learning, *Inf. Softw. Technol.* (2025) 107826.
- [18] Fuming Qu, Jinhai Liu, Xiaoyuan Liu, Lin Jiang, A multi-fault detection method with improved triplet loss based on hard sample mining, *IEEE Trans. Sustain. Energy* 12 (1) (2020) 127–137.
- [19] Xin Cai, Li Liu, Lei Zhu, Huaxiang Zhang, Dual-modality hard mining triplet-center loss for visible infrared person re-identification, *Knowl.-Based Syst.* 215 (2021) 106772.
- [20] CWE Community - CWE Abstract Type (CWE Glossary), 2021, <https://cwe.mitre.org/documents/glossary/>.
- [21] Alexander Hermans, Lucas Beyer, Bastian Leibe, In defense of the triplet loss for person re-identification, 2017, arXiv preprint [arXiv:1703.07737](https://arxiv.org/abs/1703.07737).
- [22] Jiahao Fan, Yi Li, Shaohua Wang, Tien N. Nguyen, A C/C++ code vulnerability dataset with code changes and CVE summaries, in: *Proceedings of the 17th International Conference on Mining Software Repositories*, 2020, pp. 508–512.
- [23] Zhou Yang, Jieke Shi, Junda He, David Lo, Natural attack for pre-trained models of code, in: *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1482–1493.
- [24] Yulong Yang, Haoran Fan, Chenhao Lin, Qian Li, Zhengyu Zhao, Chao Shen, Exploiting the adversarial example vulnerability of transfer learning of source code, *IEEE Trans. Inf. Forensics Secur.* (2024).
- [25] Van-Anh Nguyen, Dai Quoc Nguyen, Van Nguyen, Trung Le, Quan Hung Tran, Dinh Phung, Regvd: Revisiting graph neural networks for vulnerability detection, in: *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, 2022, pp. 178–182.
- [26] Yoon Kim, Convolutional neural networks for sentence classification, in: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2014, pp. 1746–1751.
- [27] Xiao Yu, Guancheng Lin, Xing Hu, Jacky Wai Keung, Xin Xia, Less is more: Unlocking semi-supervised deep learning for vulnerability detection, *ACM Trans. Softw. Eng. Methodol.* 34 (3) (2025) 1–37.
- [28] Tianyi Wang, Shengzhi Qin, Kam Pui Chow, Towards vulnerability types classification using pure self-attention: A common weakness enumeration based approach, in: *2021 IEEE 24th International Conference on Computational Science and Engineering*, IEEE, 2021, pp. 146–153.
- [29] P.R. Vishnu, P. Vinod, Suleiman Y. Yerima, A deep learning approach for classifying vulnerability descriptions using self attention based neural network, *J. Netw. Syst. Manage.* 30 (1) (2022) 9.
- [30] Guilong Lu, Xiaolin Ju, Xiang Chen, Shaoyu Yang, Liang Chen, Hao Shen, Assessing the effectiveness of vulnerability detection via prompt tuning: An empirical study, in: *2023 30th Asia-Pacific Software Engineering Conference*, IEEE, 2023, pp. 415–424.
- [31] Chang Xu, Huaiyu Xu, Liehuang Zhu, Xiaodong Shen, Kashif Sharif, Enhanced smart contract vulnerability detection via graph neural networks: Achieving high accuracy and efficiency, *IEEE Trans. Softw. Eng.* (2025).
- [32] Geoffrey Hinton, Oriol Vinyals, Jeff Dean, Distilling the knowledge in a neural network, 2015, arXiv preprint [arXiv:1503.02531](https://arxiv.org/abs/1503.02531).
- [33] Tim Van Erven, Peter Harremoës, Rényi divergence and Kullback-Leibler divergence, *IEEE Trans. Inform. Theory* 60 (7) (2014) 3797–3820.

- [34] Rico Sennrich, Barry Haddow, Alexandra Birch, Neural machine translation of rare words with subword units, 2015, arXiv preprint [arXiv:1508.07909](https://arxiv.org/abs/1508.07909).
- [35] Pieter-Tjerk De Boer, Dirk P Kroese, Shie Mannor, Reuven Y Rubinfeld, A tutorial on the cross-entropy method, *Ann. Oper. Res.* 134 (2005) 19–67.
- [36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, Illia Polosukhin, Attention is all you need, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [37] Yonglong Tian, Dilip Krishnan, Phillip Isola, Contrastive representation distillation, 2019, arXiv preprint [arXiv:1910.10699](https://arxiv.org/abs/1910.10699).
- [38] David E. Rumelhart, Geoffrey E. Hinton, Ronald J. Williams, Learning representations by back-propagating errors, *Nature* 323 (6088) (1986) 533–536.
- [39] Florian Schroff, Dmitry Kalenichenko, James Philbin, Facenet: A unified embedding for face recognition and clustering, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 815–823.
- [40] Kaichun Yao, Hao Wang, Chuan Qin, Hengshu Zhu, Yanjun Wu, Libo Zhang, CARL: Unsupervised code-based adversarial attacks for programming language models via reinforcement learning, *ACM Trans. Softw. Eng. Methodol.* 34 (1) (2024) 1–32.
- [41] Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, in: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, 2019, pp. 4171–4186.
- [42] Kevin Clark, Minh-Thang Luong, Quoc V Le, Christopher D Manning, Electra: Pre-training text encoders as discriminators rather than generators, 2020, arXiv preprint [arXiv:2003.10555](https://arxiv.org/abs/2003.10555).
- [43] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al., Graphcodebert: Pre-training code representations with data flow, 2020, arXiv preprint [arXiv:2009.08366](https://arxiv.org/abs/2009.08366).
- [44] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi DQ Bui, Junnan Li, Steven CH Hoi, Codet5+: Open code large language models for code understanding and generation, 2023, arXiv preprint [arXiv:2305.07922](https://arxiv.org/abs/2305.07922).
- [45] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, Peter J Liu, Exploring the limits of transfer learning with a unified text-to-text transformer, *J. Mach. Learn. Res.* 21 (140) (2020) 1–67.
- [46] Saleh Soltan, Shankar Ananthakrishnan, Jack FitzGerald, Rahul Gupta, Wael Hamza, Haidar Khan, Charith Peris, Stephen Rawls, Andy Rosenbaum, Anna Rumshisky, et al., Alexatm 20b: Few-shot learning using a large-scale multilingual seq2seq model, 2022, arXiv preprint [arXiv:2208.01448](https://arxiv.org/abs/2208.01448).
- [47] Junnan Li, Ramprasaath Selvaraju, Akhilesh Gotmare, Shafiq Joty, Caiming Xiong, Steven Chu Hong Hoi, Align before fuse: Vision and language representation learning with momentum distillation, *Adv. Neural Inf. Process. Syst.* 34 (2021) 9694–9705.
- [48] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al., Codexglue: A machine learning benchmark dataset for code understanding and generation, 2021, arXiv preprint [arXiv:2102.04664](https://arxiv.org/abs/2102.04664).
- [49] Xun Wang, Xintong Han, Weilin Huang, Dengke Dong, Matthew R Scott, Multi-similarity loss with general pair weighting for deep metric learning, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 5022–5030.
- [50] Mai Lan Ha, Volker Blanz, Deep ranking with adaptive margin triplet loss, 2021, arXiv preprint [arXiv:2107.06187](https://arxiv.org/abs/2107.06187).
- [51] CWE community - 2024 CWE Top 25 Most Dangerous Software Weaknesses, 2025, <https://cwe.mitre.org/top25/index.html>.
- [52] CWE community - CWE CATEGORY: DEPRECATED: Code, 2025, <https://cwe.mitre.org/data/definitions/{17}.html>.
- [53] CWE community - CWE CATEGORY: DEPRECATED: Source Code, 2025, <https://cwe.mitre.org/data/definitions/{18}.html>.
- [54] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, Yoshua Bengio, Learning phrase representations using RNN encoder-decoder for statistical machine translation, 2014, arXiv preprint [arXiv:1406.1078](https://arxiv.org/abs/1406.1078).
- [55] Mike Schuster, Kuldip K. Paliwal, Bidirectional recurrent neural networks, *IEEE Trans. Signal Process.* 45 (11) (1997) 2673–2681.
- [56] Alex Graves, Alex Graves, Long short-term memory, Supervised Seq. Label. Recurr. Neural Networks (2012) 37–45.
- [57] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, Piotr Dollár, Focal loss for dense object detection, in: *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2980–2988.
- [58] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, Serge Belongie, Class-balanced loss based on effective number of samples, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 9268–9277.
- [59] Aditya Krishna Menon, Sadeep Jayasumana, Ankit Singh Rawat, Himanshu Jain, Andreas Veit, Sanjiv Kumar, Long-tail learning via logit adjustment, 2020, arXiv preprint [arXiv:2007.07314](https://arxiv.org/abs/2007.07314).
- [60] Yan Zhao, Weicong Chen, Xu Tan, Kai Huang, Jihong Zhu, Adaptive logit adjustment loss for long-tailed visual recognition, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, (3) 2022, pp. 3472–3480.
- [61] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al., Gemma 3 technical report, 2025, arXiv preprint [arXiv:2503.19786](https://arxiv.org/abs/2503.19786).
- [62] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoní Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al., Gpt-4 technical report, 2023, arXiv preprint [arXiv:2303.08774](https://arxiv.org/abs/2303.08774).
- [63] Michael Fu, Chakkrit Kla Tantithamthavorn, Van Nguyen, Trung Le, Chatgpt for vulnerability detection, classification, and repair: How far are we? in: *2023 30th Asia-Pacific Software Engineering Conference, APSEC, IEEE, 2023*, pp. 632–636.
- [64] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, Yizheng Chen, Vulnerability detection with code language models: How far are we?, 2024, arXiv preprint [arXiv:2403.18624](https://arxiv.org/abs/2403.18624).
- [65] Junji Yu, Honglin Shu, Michael Fu, Dong Wang, Chakkrit Tantithamthavorn, Yasutaka Kamei, Junjie Chen, A preliminary study of large language models for multilingual vulnerability detection, in: *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2025, pp. 161–168.
- [66] Zixuan Tan, Jiayuan Zhou, Xing Hu, Shengyi Pan, Kui Liu, Xin Xia, Similar but patched code considered harmful—the impact of similar but patched code on recurring vulnerability detection and how to remove them, 2024, arXiv preprint [arXiv:2412.20740](https://arxiv.org/abs/2412.20740).
- [67] Benjamin Steenhoeck, Kalpathy Sivaraman, Renata Saldivar Gonzalez, Yevhen Mohylevskyy, Roshanak Zilouchian Moghaddam, Wei Le, Closing the gap: A user study on the real-world usefulness of AI-powered vulnerability detection & repair in the IDE, 2024, arXiv preprint [arXiv:2412.14306](https://arxiv.org/abs/2412.14306).